Voyzr Source Code Appendix

File: `chat-service\src\main\java\com\voyzr\chatservice\VoyzrChatServiceApplication.java`

```java
package com.voyzr.chatservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class VoyzrChatServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(VoyzrChatServiceApplication.class, args);
    }
}
```

File: `chat-service\src\main\java\com\voyzr\chatservice\config\JwtAuthFilter.java`

```java
package com.voyzr.chatservice.config;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.extern.slf4j.Slf4j;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;
```

```
import java.io.IOException;

import java.util.List;


@Component

@Slf4j

public class JwtAuthFilter extends OncePerRequestFilter {


    @Override

    protected void doFilterInternal(HttpServletRequest request,

                                    HttpServletResponse response,

                                    FilterChain filterChain)

            throws ServletException, IOException {


        String xUserId = request.getHeader("X-User-Id");

        if (xUserId != null) {

            Long userId = Long.parseLong(xUserId);

            String email = request.getHeader("X-User-Email") != null ?
request.getHeader("X-User-Email") : "user@voyzr.com";


            UsernamePasswordAuthenticationToken authentication =

                    new UsernamePasswordAuthenticationToken(

                            userId,

                            email,

                            List.of(new SimpleGrantedAuthority("ROLE_USER"))

                    );

            SecurityContextHolder.getContext().setAuthentication(authentication);

        }


        filterChain.doFilter(request, response);

    }

}


```

File: `chat-service\src\main\java\com\voyzr\chatservice\config\MongoConfig.java`

```java
package com.voyzr.chatservice.config;

import com.voyzr.chatservice.entity.ChatMessage;
import org.bson.Document;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.convert.converter.Converter;
import org.springframework.data.mongodb.core.convert.MongoCustomConversions;

import java.time.LocalDateTime;
import java.time.ZoneId;
import java.util.Arrays;
import java.util.Date;

@Configuration
public class MongoConfig {

    @Bean
    public MongoCustomConversions mongoCustomConversions() {
        // Must use a named class — lambdas don't work here because Spring Data
        // needs to resolve generic type arguments (Document → ChatMessage) via
        // reflection at startup. Lambdas cause type erasure → IllegalStateException.
        return new MongoCustomConversions(Arrays.asList(new ChatMessageReadConverter()));
    }

    /**
     * Reads a MongoDB BSON Document into a ChatMessage, handling BOTH:
     * - Old documents that stored the field as "ai" (pre-@Field fix)
     * - New documents that store the field as "isAi" (post-@Field fix)
     */
    static class ChatMessageReadConverter implements Converter<Document, ChatMessage> {
```

```
        @Override
        public ChatMessage convert(Document doc) {
            ChatMessage msg = new ChatMessage();

            // _id can be ObjectId or String
            Object rawId = doc.get("_id");
            msg.setId(rawId != null ? rawId.toString() : null);

            // tripId: stored as Long or Int32
            Long tripId = null;
            if (doc.getLong("tripId") != null) {
                tripId = doc.getLong("tripId");
            } else if (doc.getInteger("tripId") != null) {
                tripId = doc.getInteger("tripId").longValue();
            }
            msg.setTripId(tripId);

            msg.setSenderName(doc.getString("senderName"));
            msg.setSenderEmail(doc.getString("senderEmail"));
            msg.setContent(doc.getString("content"));

            // Timestamp: MongoDB stores LocalDateTime as a Date
            Object tsObj = doc.get("timestamp");
            if (tsObj instanceof Date date) {
msg.setTimestamp(date.toInstant().atZone(ZoneId.systemDefault()).toLocalDateTime());
            } else if (tsObj instanceof LocalDateTime ldt) {
                msg.setTimestamp(ldt);
            }

            // Handle BOTH "ai" (old docs) and "isAi" (new docs)
            Boolean isAi = null;
            if (doc.get("isAi") != null) {
                isAi = doc.getBoolean("isAi");
            } else if (doc.get("ai") != null) {
```

```
                isAi = doc.getBoolean("ai");
            }
            msg.setAi(Boolean.TRUE.equals(isAi));

            return msg;
        }
    }
}

```

File: `chat-service\src\main\java\com\voyzr\chatservice\config\SecurityConfig.java`

```java
package com.voyzr.chatservice.config;

import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import
org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthFilter jwtAuthFilter;
```

```
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
                .csrf(AbstractHttpConfigurer::disable)
                .authorizeHttpRequests(auth -> auth
                        .anyRequest().permitAll()
                )
                .sessionManagement(session -> session
                        .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
                )
                .addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

File: `chat-service\src\main\java\com\voyzr\chatservice\config\WebSocketConfig.java`

```java
package com.voyzr.chatservice.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.simp.config.MessageBrokerRegistry;
import org.springframework.web.socket.config.annotation.EnableWebSocketMessageBroker;
import org.springframework.web.socket.config.annotation.StompEndpointRegistry;
import org.springframework.web.socket.config.annotation.WebSocketMessageBrokerConfigurer;
import org.springframework.web.socket.config.annotation.WebSocketTransportRegistration;

@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
```

```
    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        config.enableSimpleBroker("/topic");
        config.setApplicationDestinationPrefixes("/app");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws-voyzr")
                .setAllowedOriginPatterns("*")
                .withSockJS();
    }

    @Override
    public void configureWebSocketTransport(WebSocketTransportRegistration registration) {
        // Increase limits for large payloads (e.g. Base64 images)
        registration.setMessageSizeLimit(5 * 1024 * 1024); // 5MB
        registration.setSendBufferSizeLimit(5 * 1024 * 1024); // 5MB
        registration.setSendTimeLimit(20 * 1000); // 20s
    }
}

```

File: `chat-service\src\main\java\com\voyzr\chatservice\controller\ChatController.java`

```java
package com.voyzr.chatservice.controller;

import com.voyzr.chatservice.entity.ChatMessage;
import com.voyzr.chatservice.repository.ChatMessageRepository;
import com.voyzr.chatservice.service.AiChatService;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpStatus;
```

```
import org.springframework.http.ResponseEntity;
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.messaging.handler.annotation.Payload;
import org.springframework.messaging.simp.SimpMessagingTemplate;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDateTime;
import java.util.*;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CopyOnWriteArrayList;

@RestController
@RequestMapping("/api/chat")
@RequiredArgsConstructor
@Slf4j
public class ChatController {

    private final SimpMessagingTemplate messagingTemplate;
    private final ChatMessageRepository chatMessageRepository;
    private final AiChatService aiChatService;

    // In-memory fallback if MongoDB is down
    private static final List<ChatMessage> inMemoryHistory = new CopyOnWriteArrayList<>();

    // ── Global exception handler for this controller ──────────────────────
    // Catches any uncaught exception (e.g., MongoDB MappingException for old
    // documents that have "ai" field instead of "isAi") and returns 200 + [].
    @ExceptionHandler(Exception.class)
    public ResponseEntity<List<ChatMessage>> handleAnyException(Exception e) {
        log.error("[ChatController] Unhandled exception — returning safe empty list: {}",
e.getMessage());
        // Return in-memory if available, otherwise empty list — never 500
        List<ChatMessage> safe = inMemoryHistory.isEmpty() ? Collections.emptyList() :
inMemoryHistory;
        return ResponseEntity.ok(safe);
```

```
    }


    @GetMapping("/ping")
    public String ping() {
        return "Chat Service is Online";
    }


    @MessageMapping("/chat.send")
    public void sendMessage(@Payload ChatMessage chatMessage) {
        log.info("Received chat message from {} for trip {}", chatMessage.getSenderName(),
chatMessage.getTripId());
        chatMessage.setTimestamp(LocalDateTime.now());
        chatMessage.setAi(false);


        try {
            ChatMessage savedMessage = chatMessageRepository.save(chatMessage);
            chatMessage = savedMessage;
        } catch (Exception e) {
            log.warn("Failed to save to MongoDB, using in-memory: {}", e.getMessage());
            if (chatMessage.getId() == null) {
                chatMessage.setId(java.util.UUID.randomUUID().toString());
            }
            inMemoryHistory.add(chatMessage);
        }


        messagingTemplate.convertAndSend("/topic/chat/" + chatMessage.getTripId(),
chatMessage);


        final ChatMessage lambdaMsg = chatMessage;
        String content = chatMessage.getContent().toLowerCase();
        if (content.contains("ai") || content.contains("attraction") ||
content.contains("place")
                || content.contains("visit") || content.contains("@")) {
            CompletableFuture.runAsync(() -> {
                try {
                    Thread.sleep(1000);
```

```
                    aiChatService.processAiResponse(lambdaMsg);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                } catch (Exception e) {
                    log.error("AI processing error: {}", e.getMessage());
                }
            });
        }
    }

    @MessageMapping("/chat.typing")
    public void sendTypingStatus(@Payload Map<String, Object> typingData) {
        String tripId = String.valueOf(typingData.get("tripId"));
        messagingTemplate.convertAndSend("/topic/chat/" + tripId + "/typing", typingData);
    }

    @PostMapping("/messages/{id}/react")
    public ResponseEntity<ChatMessage> updateReaction(@PathVariable String id, @RequestBody
Map<String, Object> reactionData) {
        log.info("Updating reaction for message {}. Data: {}", id, reactionData);
        try {
            Optional<ChatMessage> messageOpt = chatMessageRepository.findById(id);
            if (messageOpt.isEmpty()) {
                messageOpt = inMemoryHistory.stream().filter(m ->
id.equals(m.getId())).findFirst();
            }

            ChatMessage message = messageOpt.orElseThrow(() -> new
RuntimeException("Message not found with id: " + id));

            String emoji = (String) reactionData.get("emoji");
            String voterEmail = (String) reactionData.get("voterEmail");

            if (emoji == null || voterEmail == null) {
                log.warn("Invalid reaction request: emoji or voterEmail is null");
                return ResponseEntity.badRequest().build();
```

```
            }

            Map<String, List<String>> reactions = message.getReactions();
            if (reactions == null) {
                reactions = new HashMap<>();
            }

            List<String> voters = reactions.get(emoji);
            if (voters == null) {
                voters = new ArrayList<>();
                reactions.put(emoji, voters);
            } else {
                voters = new ArrayList<>(voters);
                reactions.put(emoji, voters);
            }

            if (voters.contains(voterEmail)) {
                voters.remove(voterEmail);
                log.info("Removed reaction {} from user {} on message {}", emoji,
voterEmail, id);
                if (voters.isEmpty()) {
                    reactions.remove(emoji);
                }
            } else {
                voters.add(voterEmail);
                log.info("Added reaction {} from user {} on message {}", emoji, voterEmail,
id);
            }

            message.setReactions(reactions);

            // Try to save to MongoDB, but don't fail if it's an in-memory message
            try {
                chatMessageRepository.save(message);
            } catch (Exception e) {
```

```
                log.warn("Failed to save reaction to MongoDB, it stays in-memory: {}",
e.getMessage());
            }

            // Broadcast via WebSocket
            messagingTemplate.convertAndSend("/topic/chat/" + message.getTripId(),
message);

            return ResponseEntity.ok(message);
        } catch (Exception e) {
            log.error("Failed to update reaction for message {}: {}", id, e.getMessage(),
e);
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
        }
    }

    /**
     * Returns chat history for a given tripId.
     * Falls back to in-memory (then empty list) if MongoDB fails — never throws.
     */
    @GetMapping("/history/{tripId}")
    public ResponseEntity<List<ChatMessage>> getChatHistory(@PathVariable String tripId) {
        log.info("Fetching chat history for trip: {}", tripId);
        try {
            Long id = Long.parseLong(tripId);
            List<ChatMessage> history =
chatMessageRepository.findByTripIdOrderByTimestampAsc(id);
            log.info("Found {} messages for trip {}", history.size(), id);
            return ResponseEntity.ok(history);
        } catch (NumberFormatException e) {
            log.warn("Non-numeric tripId '{}' — returning empty list", tripId);
            return ResponseEntity.ok(Collections.emptyList());
        } catch (Exception e) {
            log.error("Failed to fetch history from MongoDB for trip {}: {}", tripId,
e.getMessage());
            // If MongoDB is down or has bad docs, fall through to in-memory / empty
```

```
            List<ChatMessage> fallback = inMemoryHistory.stream()
                    .filter(m -> tripId.equals(String.valueOf(m.getTripId())))
                    .toList();
            return ResponseEntity.ok(fallback);
        }
    }

    @PostMapping("/messages/{id}/vote")
    public ResponseEntity<ChatMessage> updateVote(@PathVariable String id, @RequestBody
Map<String, Object> voteData) {
        log.info("Updating vote for message {}", id);
        try {
            Optional<ChatMessage> messageOpt = chatMessageRepository.findById(id);
            if (messageOpt.isEmpty()) {
                messageOpt = inMemoryHistory.stream().filter(m ->
id.equals(m.getId())).findFirst();
            }

            ChatMessage message = messageOpt.orElseThrow(() -> new
RuntimeException("Message not found"));

            Integer optionIndex = (Integer) voteData.get("optionIndex");
            String voterEmail = (String) voteData.get("voterEmail");

            Map<String, Object> metadata = message.getMetadata();
            if (metadata == null) metadata = new HashMap<>();

            List<Map<String, Object>> options = (List<Map<String, Object>>)
metadata.get("options");
            if (options != null && optionIndex != null && optionIndex < options.size()) {
                for (Map<String, Object> opt : options) {
                    List<String> voters = (List<String>) opt.get("voters");
                    if (voters != null) {
                        voters.remove(voterEmail);
                    }
                }
```

```
                Map<String, Object> selectedOption = options.get(optionIndex);
                List<String> voters = (List<String>) selectedOption.get("voters");
                if (voters == null) {
                    voters = new ArrayList<>();
                    selectedOption.put("voters", voters);
                }
                voters.add(voterEmail);
            }

            message.setMetadata(metadata);

            try {
                chatMessageRepository.save(message);
            } catch (Exception e) {
                log.warn("Failed to save vote to MongoDB, stays in-memory: {}",
e.getMessage());
            }

            messagingTemplate.convertAndSend("/topic/chat/" + message.getTripId(),
message);

            return ResponseEntity.ok(message);
        } catch (Exception e) {
            log.error("Failed to update vote: {}", e.getMessage());
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
        }
    }

    @PostMapping("/ai")
    public ResponseEntity<Void> chatWithAi(@RequestBody ChatMessage chatMessage) {
        log.info("Direct AI Assistant message: {}", chatMessage.getContent());
        chatMessage.setTimestamp(LocalDateTime.now());
        chatMessage.setAi(false);
        chatMessage.setTripId(null);
```

```
        try {
            ChatMessage savedMessage = chatMessageRepository.save(chatMessage);
            chatMessage = savedMessage;
        } catch (Exception e) {
            log.warn("Database error, continuing in-memory: {}", e.getMessage());
            if (chatMessage.getId() == null) {
                chatMessage.setId(java.util.UUID.randomUUID().toString());
            }
            inMemoryHistory.add(chatMessage);
        }

        messagingTemplate.convertAndSend("/topic/chat/global", chatMessage);

        final ChatMessage lambdaMsg = chatMessage;
        CompletableFuture.runAsync(() -> {
            try {
                Thread.sleep(1000);
                aiChatService.processAiResponse(lambdaMsg);
            } catch (Exception e) {
                log.error("AI processing error: {}", e.getMessage());
            }
        });

        return ResponseEntity.ok().build();
    }

    public void addToInMemory(ChatMessage msg) {
        inMemoryHistory.add(msg);
    }
}
```

File: `chat-service\src\main\java\com\voyzr\chatservice\entity\ChatMessage.java`

```java
package com.voyzr.chatservice.entity;

import com.fasterxml.jackson.annotation.JsonAlias;
import com.fasterxml.jackson.annotation.JsonProperty;
import lombok.*;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.mongodb.core.mapping.Field;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;
import java.util.HashMap;

@Document(collection = "chat_messages")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class ChatMessage {

    @Id
    private String id;

    private Long tripId;
    private String senderName;
    private String senderEmail;
    private String content;
    private LocalDateTime timestamp;

    // TEXT, IMAGE, SYSTEM, POLL, LINK
    private String type;
    private String imageUrl;
```

```
    // Store extra structured data like votes, link previews, sandbox data, auto-calculations
    private Map<String, Object> metadata = new HashMap<>();

    private Map<String, List<String>> reactions = new HashMap<>(); // emoji -> list of user emails

    /**
     * NOTE: Lombok @Data generates isAi() getter for boolean fields, causing:
     * - Spring Data MongoDB to store this as field "ai" (not "isAi")
     * - Jackson HTTP serializer to emit "isAi" key via @JsonProperty
     *
     * @Field("isAi") ensures MongoDB stores/reads using key "isAi".
     * @JsonProperty("isAi") ensures HTTP JSON uses key "isAi".
     * @JsonAlias("ai") accepts old HTTP responses that used the "ai" key.
     */
    @Field("isAi")
    @JsonProperty("isAi")
    @JsonAlias("ai")
    private boolean isAi;
}

```

File: `chat-service\src\main\java\com\voyzr\chatservice\service\AiChatService.java`

```java
package com.voyzr.chatservice.service;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.voyzr.chatservice.entity.ChatMessage;
import com.voyzr.chatservice.repository.ChatMessageRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
```

```
import org.springframework.beans.factory.annotation.Value;

import org.springframework.messaging.simp.SimpMessagingTemplate;

import org.springframework.stereotype.Service;

import org.springframework.web.client.RestTemplate;


import java.time.LocalDateTime;

import java.util.HashMap;

import java.util.Map;


@Service

@RequiredArgsConstructor

@Slf4j

public class AiChatService {


    private final ChatMessageRepository chatMessageRepository;

    private final SimpMessagingTemplate messagingTemplate;

    private final RestTemplate restTemplate = new RestTemplate();

    private final ObjectMapper objectMapper = new ObjectMapper();


    @Value("${groq.api.key:REPLACE_WITH_YOUR_GROQ_KEY}")

    private String groqApiKey;


    // Internal URLs for microservices

    private final String TRIP_SERVICE_URL = "http://localhost:8081/api/trips/";

    private final String PLACES_SERVICE_URL =
"http://localhost:8087/api/places/attractions";


    public void processAiResponse(ChatMessage userMessage) {

        log.info("Processing AI response for message: '{}', tripId: {}",
userMessage.getContent(), userMessage.getTripId());

        String content = userMessage.getContent().trim();


        // Persona detection: @bot:historian, @bot:budget, @bot:adventurer

        String persona = "default";

        String cleanContent = content;
```

```
        String lowContent = content.toLowerCase();


        if (lowContent.startsWith("@bot:")) {
            int firstSpace = content.indexOf(" ");
            if (firstSpace != -1) {
                persona = content.substring(5, firstSpace).toLowerCase();
                cleanContent = content.substring(firstSpace).trim();
            } else {
                persona = content.substring(5).toLowerCase();
                cleanContent = "";
            }
            log.info("Detected persona: {}, clean content: '{}'", persona, cleanContent);
        } else if (content.startsWith("@")) {
            // General @ mention or file path mention
            persona = "default";
            cleanContent = content;
            log.info("Detected general @ mention, content: '{}'", cleanContent);
        }


        // If tripId is null, it's a general travel question
        if (userMessage.getTripId() == null) {
            log.info("TripId is null, generating general Groq response");
            generateGroqResponse(userMessage, persona, cleanContent);
            return;
        }


        // Check if user is asking for attractions for a specific trip
        String lowClean = cleanContent.toLowerCase();
        if (lowClean.contains("attraction") || lowClean.contains("place") ||
lowClean.contains("visit") || lowClean.contains("see")) {
            log.info("Attraction keywords detected for trip {}, fetching attractions...",
userMessage.getTripId());
            fetchAndSendAttractions(userMessage.getTripId(), persona);
        } else {
            log.info("No attraction keywords, generating conversational Groq response for
trip {}", userMessage.getTripId());
```

```
            generateGroqResponse(userMessage, persona, cleanContent);
        }
    }

    private String getPersonaPrompt(String persona) {
        switch (persona) {
            case "historian":
                return "You are the Historian Guide. Focus on deep historical facts, architecture, and cultural significance. Use a sophisticated, knowledgeable tone.";
            case "budget":
                return "You are the Budget Buddy. Focus on free attractions, affordable street food, and saving money. Use a friendly, practical, and savvy tone.";
            case "adventurer":
                return "You are the Adventurer. Focus on hidden gems, off-beat paths, hiking, and thrill-seeking activities. Use an energetic and daring tone.";
            default:
                return "You are Voyzr AI, a helpful travel assistant for a group trip. Keep responses concise and friendly.";
        }
    }

    private void generateGroqResponse(ChatMessage userMessage, String persona, String cleanContent) {
        log.info("Requesting Groq response for persona: {}, content: '{}'", persona, cleanContent);
        if ("REPLACE_WITH_YOUR_GROQ_KEY".equals(groqApiKey) || groqApiKey == null || groqApiKey.isEmpty()) {
            log.warn("Groq API key is missing or default");
            sendAiMessage(userMessage.getTripId(), "I'm your Voyzr AI assistant! Please add a Groq API key to my configuration to unlock full conversational powers. You can still ask me about 'attractions'!");
            return;
        }

        try {
            String url = "https://api.groq.com/openai/v1/chat/completions";
            String systemPrompt = getPersonaPrompt(persona);
```

```
            // Basic JSON escaping
            String escapedContent = cleanContent.replace("\\", "\\\\").replace("\"",
"\\\"").replace("\n", "\\n");

            String requestJson = String.format(
                "{\"model\": \"llama-3.1-8b-instant\", \"messages\": [" +
                "{\"role\": \"system\", \"content\": \"%s\"}," +
                "{\"role\": \"user\", \"content\": \"%s\"}" +
                "], \"max_tokens\": 512, \"temperature\": 0.7}",
                systemPrompt.replace("\"", "\\\""),
                escapedContent
            );

            org.springframework.http.HttpHeaders headers = new
org.springframework.http.HttpHeaders();
            headers.setContentType(org.springframework.http.MediaType.APPLICATION_JSON);
            headers.setBearerAuth(groqApiKey);

            org.springframework.http.HttpEntity<String> entity = new
org.springframework.http.HttpEntity<>(requestJson, headers);
            String response = restTemplate.postForObject(url, entity, String.class);

            JsonNode root = objectMapper.readTree(response);
            String aiText =
root.path("choices").get(0).path("message").path("content").asText();

            log.info("Received Groq response successfully");
            sendAiMessage(userMessage.getTripId(), aiText);
        } catch (Exception e) {
            log.error("Groq API call failed: {}", e.getMessage(), e);
            sendAiMessage(userMessage.getTripId(), "I'm a bit overwhelmed right now. Can
you try asking that again?");
        }
    }

    private void sendAiMessage(Long tripId, String content) {
```

```
        sendAiMessage(tripId, content, "TEXT", null);
    }

    private void sendAiMessage(Long tripId, String content, String type, Map<String,
Object> metadata) {
        log.info("Sending AI message. tripId={}, type={}, contentLen={}", tripId, type,
content.length());
        ChatMessage aiMessage = ChatMessage.builder()
                .tripId(tripId)
                .senderName("Voyzr AI")
                .senderEmail("ai@voyzr.com")
                .content(content)
                .timestamp(LocalDateTime.now())
                .isAi(true)
                .type(type)
                .metadata(metadata != null ? metadata : new HashMap<>())
                .build();

        try {
            chatMessageRepository.save(aiMessage);
        } catch (Exception e) {
            log.warn("Failed to save AI message to DB: {}", e.getMessage());
        }

        String topic = (tripId == null) ? "/topic/chat/global" : "/topic/chat/" + tripId;
        log.info("Broadcasting to topic: {}", topic);
        messagingTemplate.convertAndSend(topic, aiMessage);
    }

    private void fetchAndSendAttractions(Long tripId, String persona) {
        try {
            JsonNode tripNode = restTemplate.getForObject(TRIP_SERVICE_URL + tripId,
JsonNode.class);
            if (tripNode == null || !tripNode.has("destinationLat") ||
tripNode.get("destinationLat").isNull()) {
```

```
                sendAiMessage(tripId, "I couldn't find the exact location for this trip's
destination.");
                return;
            }

            double lat = tripNode.get("destinationLat").asDouble();
            double lng = tripNode.get("destinationLng").asDouble();
            String destination = tripNode.has("destination") ?
tripNode.get("destination").asText() : "your destination";

            String intro = "Searching for attractions...";
            if (persona.equals("historian")) intro = "Let me unearth some historical
treasures in " + destination + " for you...";
            else if (persona.equals("budget")) intro = "Finding some great free spots in "
+ destination + " to save you money!";
            else if (persona.equals("adventurer")) intro = "Scouting for epic adventures in
" + destination + "!";

            sendAiMessage(tripId, intro);

            String placesUrl = PLACES_SERVICE_URL + "?lat=" + lat + "&lng=" + lng;
            JsonNode placesResponse = restTemplate.getForObject(placesUrl, JsonNode.class);

            if (placesResponse != null && placesResponse.isArray() &&
!placesResponse.isEmpty()) {
                int count = 0;
                for (JsonNode place : placesResponse) {
                    if (count >= 3) break;

                    String name = place.has("name") ? place.get("name").asText() : "Unknown
Place";
                    String address = place.has("address") ? place.get("address").asText() :
"";
                    double pLat = place.has("lat") ? place.get("lat").asDouble() : lat;
                    double pLng = place.has("lng") ? place.get("lng").asDouble() : lng;

                    // Use real database image if available
```

```
                    String photo = (place.has("imageUrl") &&
!place.get("imageUrl").asText().isEmpty())

                        ? place.get("imageUrl").asText()

                        : "https://loremflickr.com/400/300/travel," + name.replace(" ",
"");



                    Map<String, Object> metadata = new HashMap<>();

                    metadata.put("name", name);

                    metadata.put("address", address);

                    metadata.put("latitude", pLat);

                    metadata.put("longitude", pLng);

                    metadata.put("photoUrl", photo);

                    metadata.put("rating", place.has("rating") ?
place.get("rating").asDouble() : 4.5);



                    sendAiMessage(tripId, "I recommend visiting **" + name + "**. " +
address, "PLACE_CARD", metadata);

                    count++;

                }

            } else {

                sendAiMessage(tripId, "I couldn't find any specific attractions right now,
but I hope you have a great time in " + destination + "!");

            }

        } catch (Exception e) {

            log.error("Failed to fetch attractions for trip {}: {}", tripId,
e.getMessage());

            sendAiMessage(tripId, "I'm having trouble fetching attractions right now.");

        }

    }

}


```


File: `expense-
service\src\main\java\com\voyzr\expenseservice\VoyzrExpenseServiceApplication.java`


```java
```

```
package com.voyzr.expenseservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class VoyzrExpenseServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(VoyzrExpenseServiceApplication.class, args);
    }
}

```

File: `expense-service\src\main\java\com\voyzr\expenseservice\config\DatabaseMigrator.java`

```java
package com.voyzr.expenseservice.config;

import com.voyzr.expenseservice.entity.Expense;
import com.voyzr.expenseservice.repository.ExpenseRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

import java.util.List;

/**
 * Automatically fixes existing database records that are missing a userId.
 */
@Component
@Slf4j
@RequiredArgsConstructor
```

```
public class DatabaseMigrator implements CommandLineRunner {

    private final ExpenseRepository expenseRepository;

    @Override
    public void run(String... args) {
        log.info("Checking for expenses missing user IDs...");

        List<Expense> allExpenses = expenseRepository.findAll();
        long fixedCount = 0;

        for (Expense expense : allExpenses) {
            if (expense.getUserId() == null) {
                expense.setUserId(1L); // Assign to default admin user
                expenseRepository.save(expense);
                fixedCount++;
            }
        }

        if (fixedCount > 0) {
            log.info("Migration complete: Fixed {} expenses.", fixedCount);
        } else {
            log.info("No migration needed.");
        }
    }
}

```

File: `expense-service\src\main\java\com\voyzr\expenseservice\config\JwtAuthFilter.java`

```java
package com.voyzr.expenseservice.config;

import jakarta.servlet.FilterChain;
```

```
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.extern.slf4j.Slf4j;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.List;

@Component
@Slf4j
public class JwtAuthFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
            throws ServletException, IOException {

        String xUserId = request.getHeader("X-User-Id");
        if (xUserId != null) {
            Long userId = Long.parseLong(xUserId);
            String email = request.getHeader("X-User-Email") != null ?
request.getHeader("X-User-Email") : "user@voyzr.com";

            UsernamePasswordAuthenticationToken authentication =
                    new UsernamePasswordAuthenticationToken(
                            userId,
                            email,
                            List.of(new SimpleGrantedAuthority("ROLE_USER"))
                    );
```

```
            SecurityContextHolder.getContext().setAuthentication(authentication);
        }

        filterChain.doFilter(request, response);
    }
}

```

File: `expense-service\src\main\java\com\voyzr\expenseservice\config\SecurityConfig.java`

```java
package com.voyzr.expenseservice.config;

import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthFilter jwtAuthFilter;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
```

```
        http
                .csrf(AbstractHttpConfigurer::disable)
                .authorizeHttpRequests(auth -> auth
                        .anyRequest().permitAll() // Gateway handles primary auth
                )
                .sessionManagement(session -> session
                        .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
                )
                .addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}

```

File: `expense-service\src\main\java\com\voyzr\expenseservice\controller\ExpenseController.java`

```java
package com.voyzr.expenseservice.controller;

import com.voyzr.expenseservice.entity.Expense;
import com.voyzr.expenseservice.entity.ExpenseSplit;
import com.voyzr.expenseservice.service.ExpenseService;
import jakarta.servlet.http.HttpServletRequest;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Collections;
import java.util.List;
```

```
@RestController
@RequestMapping("/api/expenses")
@RequiredArgsConstructor
@Slf4j
public class ExpenseController {

    private final ExpenseService expenseService;

    @GetMapping("/trip/{tripId}")
    public ResponseEntity<List<Expense>> getExpenses(@PathVariable Long tripId) {
        log.info("GET /api/expenses/trip/{} - Fetching expenses", tripId);
        return ResponseEntity.ok(expenseService.getExpensesByTrip(tripId));
    }

    @GetMapping("/bulk")
    public ResponseEntity<List<Expense>> getExpensesByTrips(@RequestParam(name = "ids")
List<Long> ids) {
        log.info("GET /api/expenses/bulk?ids={} - Fetching bulk expenses", ids);
        if (ids == null || ids.isEmpty()) {
            return ResponseEntity.ok(Collections.emptyList());
        }
        return ResponseEntity.ok(expenseService.getExpensesByTrips(ids));
    }

    @PostMapping
    public ResponseEntity<Expense> addExpense(@RequestBody Expense expense,
HttpServletRequest request) {
        log.info("POST /api/expenses - Adding new expense: {}", expense.getDescription());
        // Gateway injects X-User-Id header after validating JWT
        String xUserId = request.getHeader("X-User-Id");
        if (xUserId != null) {
            try {
                expense.setUserId(Long.parseLong(xUserId));
            } catch (NumberFormatException e) {
                log.warn("Could not parse X-User-Id header: {}", xUserId);
```

```
            expense.setUserId(1L); // fallback
        }
    } else {
        expense.setUserId(1L); // fallback for local testing
    }
    return ResponseEntity.ok(expenseService.addExpense(expense));
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteExpense(@PathVariable Long id) {
    log.info("DELETE /api/expenses/{} - Deleting expense", id);
    expenseService.deleteExpense(id);
    return ResponseEntity.ok().build();
}

// --- Enhanced Features ---

@PutMapping("/{id}/settle")
public ResponseEntity<Void> settleExpense(@PathVariable Long id) {
    log.info("PUT /api/expenses/{}/settle", id);
    expenseService.setSettlementStatus(id, Expense.SettlementStatus.SETTLED);
    return ResponseEntity.ok().build();
}

@PutMapping("/{id}/unsettle")
public ResponseEntity<Void> unsettleExpense(@PathVariable Long id) {
    log.info("PUT /api/expenses/{}/unsettle", id);
    expenseService.setSettlementStatus(id, Expense.SettlementStatus.PENDING);
    return ResponseEntity.ok().build();
}

@GetMapping("/{id}/splits")
public ResponseEntity<List<ExpenseSplit>> getSplits(@PathVariable Long id) {
    log.info("GET /api/expenses/{}/splits", id);
    return ResponseEntity.ok(expenseService.getExpenseSplits(id));
```

```
    }

    @PostMapping("/{id}/splits")
    public ResponseEntity<List<ExpenseSplit>> saveSplits(@PathVariable Long id,
@RequestBody List<ExpenseSplit> splits) {
        log.info("POST /api/expenses/{}/splits - Saving {} splits", id, splits.size());
        return ResponseEntity.ok(expenseService.saveSplits(id, splits));
    }
}
```

File: `expense-service\src\main\java\com\voyzr\expenseservice\entity\Expense.java`

```java
package com.voyzr.expenseservice.entity;

import jakarta.persistence.*;
import lombok.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Table(name = "expenses")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Expense {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
```

```
private Long tripId;

@Column(nullable = false)
private Long userId;

@Column(nullable = false)
private String description;

@Column(nullable = false)
private BigDecimal amount;

@Column(nullable = false)
private String paidBy; // display name of payer

@Column(nullable = false)
private String category;

// Optional notes / tags
@Column(columnDefinition = "TEXT")
private String notes;

// Base64 data URI of uploaded receipt image
@Column(name = "receipt_url", columnDefinition = "TEXT")
private String receiptUrl;

// Settlement tracking
@Enumerated(EnumType.STRING)
@Column(name = "settlement_status", nullable = false)
@Builder.Default
private SettlementStatus settlementStatus = SettlementStatus.PENDING;

// How the expense is split
@Enumerated(EnumType.STRING)
@Column(name = "split_type", nullable = false)
@Builder.Default
```

```
    private SplitType splitType = SplitType.EQUAL;

    @Column(nullable = false)
    private LocalDateTime timestamp;

    @PrePersist
    protected void onCreate() {
        if (timestamp == null) {
            timestamp = LocalDateTime.now();
        }
        if (settlementStatus == null) {
            settlementStatus = SettlementStatus.PENDING;
        }
        if (splitType == null) {
            splitType = SplitType.EQUAL;
        }
    }

    public enum SettlementStatus {
        PENDING, SETTLED
    }

    public enum SplitType {
        EQUAL, CUSTOM
    }

    @Transient
    private java.util.List<ExpenseSplit> splits;
}
```

File: `expense-service\src\main\java\com\voyzr\expenseservice\entity\ExpenseSplit.java`

```java
```

```
package com.voyzr.expenseservice.entity;

import jakarta.persistence.*;
import lombok.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;

/**
 * Tracks how an expense is split among trip members.
 * One row per person per expense.
 */
@Entity
@Table(
    name = "expense_splits",
    uniqueConstraints = @UniqueConstraint(columnNames = {"expense_id", "user_id"})
)
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class ExpenseSplit {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "expense_id", nullable = false)
    private Long expenseId;

    @Column(name = "user_id")
    private Long userId;

    @Column(name = "user_name", nullable = false)
    private String userName;
```

```
    @Column(name = "user_email")
    private String userEmail;

    @Column(name = "owed_amount", nullable = false)
    private BigDecimal owedAmount;

    @Column(name = "is_paid", nullable = false)
    @Builder.Default
    private Boolean isPaid = false;

    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        if (isPaid == null) isPaid = false;
    }
}
```

File: `expense-service\src\main\java\com\voyzr\expenseservice\repository\ExpenseRepository.java`

```java
package com.voyzr.expenseservice.repository;

import com.voyzr.expenseservice.entity.Expense;
import com.voyzr.expenseservice.entity.Expense.SettlementStatus;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface ExpenseRepository extends JpaRepository<Expense, Long> {
    List<Expense> findByTripIdOrderByTimestampDesc(Long tripId);
```

```
    List<Expense> findByTripIdIn(List<Long> tripIds);

    List<Expense> findByTripIdAndSettlementStatus(Long tripId, SettlementStatus status);
}

```

File: `expense-service\src\main\java\com\voyzr\expenseservice\repository\ExpenseSplitRepository.java`

```java
package com.voyzr.expenseservice.repository;

import com.voyzr.expenseservice.entity.ExpenseSplit;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.util.List;

public interface ExpenseSplitRepository extends JpaRepository<ExpenseSplit, Long> {

    List<ExpenseSplit> findByExpenseId(Long expenseId);

    List<ExpenseSplit> findByExpenseIdIn(List<Long> expenseIds);

    @Modifying
    @Query("DELETE FROM ExpenseSplit s WHERE s.expenseId = :expenseId")
    void deleteByExpenseId(@Param("expenseId") Long expenseId);
}

```

File: `expense-service\src\main\java\com\voyzr\expenseservice\service\ExpenseService.java`

```java
package com.voyzr.expenseservice.service;

import com.voyzr.expenseservice.entity.Expense;
import com.voyzr.expenseservice.entity.ExpenseSplit;
import com.voyzr.expenseservice.repository.ExpenseRepository;
import com.voyzr.expenseservice.repository.ExpenseSplitRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
@Transactional
@Slf4j
@RequiredArgsConstructor
public class ExpenseService {

    private final ExpenseRepository expenseRepository;
    private final ExpenseSplitRepository splitRepository;

    public Expense addExpense(Expense expense) {
        log.info("Adding expense for trip {}", expense.getTripId());
        Expense savedExpense = expenseRepository.save(expense);

        // If the client sent splits in the request, save them now
        // This makes the frontend POST /api/expenses call atomic
        if (expense.getSplits() != null && !expense.getSplits().isEmpty()) {
            expense.getSplits().forEach(s -> s.setExpenseId(savedExpense.getId()));
            List<ExpenseSplit> savedSplits = splitRepository.saveAll(expense.getSplits());
            savedExpense.setSplits(savedSplits);
        }
```

```
        return savedExpense;
    }

    @Transactional(readOnly = true)
    public List<Expense> getExpensesByTrip(Long tripId) {
        List<Expense> expenses =
expenseRepository.findByTripIdOrderByTimestampDesc(tripId);
        // eager load splits for UI
        expenses.forEach(e -> e.setSplits(splitRepository.findByExpenseId(e.getId())));
        return expenses;
    }

    @Transactional(readOnly = true)
    public List<Expense> getExpensesByTrips(List<Long> tripIds) {
        List<Expense> expenses = expenseRepository.findByTripIdIn(tripIds);
        // eager load splits
        expenses.forEach(e -> e.setSplits(splitRepository.findByExpenseId(e.getId())));
        return expenses;
    }

    public void deleteExpense(Long id) {
        splitRepository.deleteByExpenseId(id); // Clean up splits first
        expenseRepository.deleteById(id);
    }

    // --- New Enhanced Features ---

    public void setSettlementStatus(Long expenseId, Expense.SettlementStatus status) {
        Expense expense = expenseRepository.findById(expenseId)
                .orElseThrow(() -> new RuntimeException("Expense not found"));
        expense.setSettlementStatus(status);
        expenseRepository.save(expense);
    }
```

```
    public List<ExpenseSplit> saveSplits(Long expenseId, List<ExpenseSplit> splits) {
        // Verify expense exists
        if (!expenseRepository.existsById(expenseId)) {
            throw new RuntimeException("Expense not found");
        }

        // Remove old splits, replace with new
        splitRepository.deleteByExpenseId(expenseId);

        splits.forEach(s -> s.setExpenseId(expenseId));
        return splitRepository.saveAll(splits);
    }

    @Transactional(readOnly = true)
    public List<ExpenseSplit> getExpenseSplits(Long expenseId) {
        return splitRepository.findByExpenseId(expenseId);
    }
}

```

File: `location-service\src\main\java\com\voyzr\locationservice\VoyzrLocationServiceApplication.java`

```java
package com.voyzr.locationservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class VoyzrLocationServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(VoyzrLocationServiceApplication.class, args);
    }
```

```
}
```

File: `location-service\src\main\java\com\voyzr\locationservice\config\WebSocketConfig.java`

```java
package com.voyzr.locationservice.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.simp.config.MessageBrokerRegistry;
import org.springframework.web.socket.config.annotation.EnableWebSocketMessageBroker;
import org.springframework.web.socket.config.annotation.StompEndpointRegistry;
import org.springframework.web.socket.config.annotation.WebSocketMessageBrokerConfigurer;
import org.springframework.web.socket.config.annotation.WebSocketTransportRegistration;

@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        config.enableSimpleBroker("/topic");
        config.setApplicationDestinationPrefixes("/app");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws-location")
                .setAllowedOriginPatterns("*")
                .withSockJS();
    }

    @Override
```

```
    public void configureWebSocketTransport(WebSocketTransportRegistration registration) {
        registration.setMessageSizeLimit(1024 * 1024); // 1MB is enough for location
        registration.setSendBufferSizeLimit(1024 * 1024);
        registration.setSendTimeLimit(10 * 1000);
    }
}
```

File: `location-service\src\main\java\com\voyzr\locationservice\controller\LocationController.java`

```java
package com.voyzr.locationservice.controller;

import com.voyzr.locationservice.dto.LocationUpdate;
import com.voyzr.locationservice.entity.MemberLocation;
import com.voyzr.locationservice.repository.MemberLocationRepository;
import com.voyzr.locationservice.service.DistanceMatrixService;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.messaging.handler.annotation.Payload;
import org.springframework.messaging.simp.SimpMessagingTemplate;
import org.springframework.web.bind.annotation.*;
import org.springframework.http.ResponseEntity;
import org.springframework.http.HttpStatus;

@RestController
@RequiredArgsConstructor
@Slf4j
public class LocationController {

    private final SimpMessagingTemplate messagingTemplate;
    private final DistanceMatrixService distanceMatrixService;
```

```
    private final MemberLocationRepository repository;

    private final com.voyzr.locationservice.service.SynchronizationService syncService;


    @ExceptionHandler(Exception.class)

    public ResponseEntity<com.voyzr.locationservice.dto.SyncReport>
handleException(Exception e) {

        log.error("Unhandled error in LocationController: ", e);

        // Return an empty report with an error message instead of null

        com.voyzr.locationservice.dto.SyncReport errorReport =
com.voyzr.locationservice.dto.SyncReport.builder()

                .destinationName("Error: " + e.getMessage())

                .memberStatus(new java.util.ArrayList<>())

                .build();

        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(errorReport);

    }


    @MessageMapping("/location.update")

    public void updateLocation(@Payload LocationUpdate update) {

        String type = update.getType() != null ? update.getType() : "LOCATION";


        log.debug("Location update [{}] for trip {}: User {} at {},{}",

                type, update.getTripId(), update.getUserId(), update.getLat(),
update.getLng());


        // 1. Persist only if it's a real location update

        if ("LOCATION".equals(type)) {

            MemberLocation loc =
repository.findFirstByTripIdAndUserIdOrderByUpdatedAtDesc(update.getTripId(),
update.getUserId())

                    .orElse(MemberLocation.builder()

                            .tripId(update.getTripId())

                            .userId(update.getUserId())

                            .build());


            loc.setLat(update.getLat());

            loc.setLng(update.getLng());
```

```
            loc.setStatus(update.getStatus());
            repository.save(loc);
        }

        // 2. Broadcast to all members of this trip (including CURSOR movements)
        messagingTemplate.convertAndSend("/topic/location/" + update.getTripId(), update);
    }
    @GetMapping("/api/location/eta")
    public DistanceMatrixService.DurationResult getEta(
            @RequestParam Double originLat, @RequestParam Double originLng,
            @RequestParam Double destLat, @RequestParam Double destLng) {
        return distanceMatrixService.getDuration(originLat, originLng, destLat, destLng);
    }

    @GetMapping("/api/location/sync/{tripId}")
    public com.voyzr.locationservice.dto.SyncReport getTripSync(@PathVariable Long tripId)
{
        return syncService.calculateSync(tripId);
    }
}

```

File: `location-service\src\main\java\com\voyzr\locationservice\dto\LocationUpdate.java`

```java
package com.voyzr.locationservice.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
```

```
@AllArgsConstructor
@Builder
public class LocationUpdate {
    private Long tripId;
    private Long userId;
    private String userName;
    private Double lat;
    private Double lng;
    private String status;
    private String type; // LOCATION, CURSOR, RALLY_POINT
}
```

File: `location-service\src\main\java\com\voyzr\locationservice\dto\SyncReport.java`

```java
package com.voyzr.locationservice.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import java.time.LocalDateTime;
import java.util.List;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class SyncReport {
    private Long tripId;
    private String destinationName;
    private LocalDateTime targetArrivalTime;
    private List<MemberSyncStatus> memberStatus;
```

```
    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    @Builder
    public static class MemberSyncStatus {
        private Long userId;
        private String userName;
        private String currentEtaText;
        private LocalDateTime recommendedStartTime;
        private String syncMessage; // e.g., "Leave now to arrive on time"
    }
}
```

File: `location-service\src\main\java\com\voyzr\locationservice\entity\MemberLocation.java`

```java
package com.voyzr.locationservice.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "member_locations")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class MemberLocation {

    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private Long tripId;

    @Column(nullable = false)
    private Long userId;

    private Double lat;
    private Double lng;

    private String status; // e.g., "STATIONARY", "MOVING"

    private LocalDateTime updatedAt;

    @PrePersist
    @PreUpdate
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }
}

```

File: `location-service\src\main\java\com\voyzr\locationservice\repository\MemberLocationRepository.java`

```java
package com.voyzr.locationservice.repository;

import com.voyzr.locationservice.entity.MemberLocation;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;
import java.util.Optional;
```

```
public interface MemberLocationRepository extends JpaRepository<MemberLocation, Long> {

    List<MemberLocation> findByTripId(Long tripId);

    Optional<MemberLocation> findFirstByTripIdAndUserIdOrderByUpdatedAtDesc(Long tripId,
Long userId);

}


```


File: `location-service\src\main\java\com\voyzr\locationservice\service\DistanceMatrixService.java`


```java
package com.voyzr.locationservice.service;


import com.fasterxml.jackson.databind.JsonNode;

import com.fasterxml.jackson.databind.ObjectMapper;

import lombok.AllArgsConstructor;

import lombok.Data;

import lombok.RequiredArgsConstructor;

import lombok.extern.slf4j.Slf4j;

import org.springframework.beans.factory.annotation.Value;

import org.springframework.stereotype.Service;

import org.springframework.web.client.RestTemplate;


import java.time.LocalDateTime;

import java.util.Map;

import java.util.concurrent.ConcurrentHashMap;


@Service

@Slf4j

@RequiredArgsConstructor

public class DistanceMatrixService {


    @Value("${geoapify.routing-key}")
```

```
    private String geoapifyKey;

    private final RestTemplate restTemplate = new RestTemplate();
    private final ObjectMapper objectMapper = new ObjectMapper();

    // Cache to prevent redundant Geoapify calls for the same route within a short period
    private final Map<String, CachedDuration> cache = new ConcurrentHashMap<>();
    private static final long CACHE_TTL_MINUTES = 10;

    @Data
    @AllArgsConstructor
    private static class CachedDuration {
        private DurationResult result;
        private LocalDateTime expiry;

        public boolean isExpired() {
            return LocalDateTime.now().isAfter(expiry);
        }
    }

    public DurationResult getDuration(double originLat, double originLng, double destLat,
double destLng) {
        // Create a cache key using rounded coordinates to increase cache hit rate (approx
10km grid)
        String cacheKey = String.format("%.1f,%.1f|%.1f,%.1f", originLat, originLng,
destLat, destLng);

        CachedDuration cached = cache.get(cacheKey);
        if (cached != null && !cached.isExpired()) {
            log.debug("Using cached duration for {}", cacheKey);
            return cached.getResult();
        }

        // Geoapify Routing API
        String url = String.format(
```

```
"https://api.geoapify.com/v1/routing?waypoints=%f,%f|%f,%f&mode=drive&apiKey=%s",
                originLat, originLng, destLat, destLng, geoapifyKey);

        try {
            String response = restTemplate.getForObject(url, String.class);
            JsonNode root = objectMapper.readTree(response);

            JsonNode feature = root.path("features").get(0);
            if (feature != null && !feature.isMissingNode()) {
                long seconds = feature.path("properties").path("time").asLong();

                // Format duration text manually
                long hours = seconds / 3600;
                long minutes = (seconds % 3600) / 60;
                String text = hours > 0 ? hours + "h " + minutes + "m" : minutes + "m";

                DurationResult result = new DurationResult(seconds, text);
                cache.put(cacheKey, new CachedDuration(result,
LocalDateTime.now().plusMinutes(CACHE_TTL_MINUTES)));
                return result;
            }
        } catch (Exception e) {
            log.error("Error fetching Geoapify routing: {}", e.getMessage());
        }
        return null;
    }

    @Data
    @AllArgsConstructor
    public static class DurationResult {
        private long seconds;
        private String text;
    }
}
```

```

## File: `location-service\src\main\java\com\voyzr\locationservice\service\SynchronizationService.java`

```java
package com.voyzr.locationservice.service;

import com.voyzr.locationservice.dto.SyncReport;
import com.voyzr.locationservice.entity.MemberLocation;
import com.voyzr.locationservice.repository.MemberLocationRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

@Service
@Slf4j
@RequiredArgsConstructor
public class SynchronizationService {

    private final MemberLocationRepository locationRepository;
    private final DistanceMatrixService distanceMatrixService;
    private final RestTemplate restTemplate = new RestTemplate();

    // Cache to prevent slamming Trip Service during frequent location updates
    private final Map<Long, CachedTrip> tripCache = new ConcurrentHashMap<>();
    private static final long CACHE_TTL_SECONDS = 120; // 2 minutes
```
```

```
private static class CachedTrip {
    Map<String, Object> data;
    LocalDateTime expiry;

    CachedTrip(Map<String, Object> data) {
        this.data = data;
        this.expiry = LocalDateTime.now().plusSeconds(CACHE_TTL_SECONDS);
    }

    boolean isExpired() {
        return LocalDateTime.now().isAfter(expiry);
    }
}

// Partial DTO to map Trip Service response
public static class InternalTripResponse {
    public Long id;
    public String destination;
    public Double destinationLat;
    public Double destinationLng;
    public LocalDateTime targetArrivalTime;
    public List<InternalMember> members;
}

public static class InternalMember {
    public Long userId;
    public String userName;
}

public SyncReport calculateSync(Long tripId) {
    log.info("Calculating arrival synchronization for trip {}", tripId);

    // 1. Fetch Trip details from Trip Service (with simple caching)
    Map<String, Object> data = null;
```

```
        CachedTrip cached = tripCache.get(tripId);

        if (cached != null && !cached.isExpired()) {
            data = cached.data;
            log.debug("Using cached trip details for trip {}", tripId);
        } else {
            String tripUrl = "http://localhost:8081/api/trips/" + tripId;
            try {
                @SuppressWarnings("unchecked")
                Map<String, Object> response = restTemplate.getForObject(tripUrl,
Map.class);
                if (response != null && response.containsKey("data")) {
                    data = (Map<String, Object>) response.get("data");
                    tripCache.put(tripId, new CachedTrip(data));
                }
            } catch (Exception e) {
                log.error("Failed to fetch trip details from {}: {}", tripUrl,
e.getMessage());
                // If service is down but we have expired cache, use it as fallback
                if (cached != null) {
                    log.warn("Trip Service unreachable, falling back to EXPIRED cache for
trip {}", tripId);
                    data = cached.data;
                } else {
                    throw new RuntimeException("Trip Service Unreachable: " +
e.getMessage());
                }
            }
        }

        if (data == null) {
            log.error("Trip service returned null or invalid data for trip {}", tripId);
            throw new RuntimeException("Invalid response from Trip Service");
        }

        String destinationName = (String) data.get("destination");
```

```
        // Safer numeric casting
        Double destLat = data.get("destinationLat") != null ? ((Number)
data.get("destinationLat")).doubleValue() : null;
        Double destLng = data.get("destinationLng") != null ? ((Number)
data.get("destinationLng")).doubleValue() : null;

        // Handle coordinate fallback
        if (destLat == null) destLat = 20.5937; // Fallback to India center
        if (destLng == null) destLng = 78.9629;

        // More robust date parsing for targetArrivalTime
        LocalDateTime targetTime = null;
        Object targetTimeObj = data.get("targetArrivalTime");
        if (targetTimeObj instanceof List) {
            try {
                // Jackson often sends LocalDateTime as [year, month, day, hour, minute]
                List<Integer> parts = (List<Integer>) targetTimeObj;
                if (parts.size() >= 5) {
                    targetTime = LocalDateTime.of(parts.get(0), parts.get(1), parts.get(2),
parts.get(3), parts.get(4));
                }
            } catch (Exception e) {
                log.warn("Failed to parse target time list {}, defaulting", targetTimeObj);
            }
        } else if (targetTimeObj != null) {
            try {
                String timeStr = targetTimeObj.toString().replace(" ", "T");
                targetTime = LocalDateTime.parse(timeStr);
            } catch (Exception e) {
                log.warn("Failed to parse target time string '{}', defaulting",
targetTimeObj);
            }
        }

        if (targetTime == null) {
```

```
            targetTime = LocalDateTime.now().plusDays(1).withHour(10).withMinute(0);
        }

        List<Map<String, Object>> members = (List<Map<String, Object>>)
data.get("members");
        if (members == null) members = new ArrayList<>();

        SyncReport report = SyncReport.builder()
                .tripId(tripId)
                .destinationName(destinationName)
                .targetArrivalTime(targetTime)
                .memberStatus(new ArrayList<>())
                .build();

        // 2. For each member, calculate sync
        final Double finalDestLat = destLat;
        final Double finalDestLng = destLng;
        final LocalDateTime finalTargetTime = targetTime;

        for (Map<String, Object> m : members) {
            Long userId = ((Number) m.get("userId")).longValue();
            String userName = (String) m.get("userName");

            MemberLocation loc =
locationRepository.findFirstByTripIdAndUserIdOrderByUpdatedAtDesc(tripId,
userId).orElse(null);

            if (loc == null || loc.getLat() == null) {
                report.getMemberStatus().add(SyncReport.MemberSyncStatus.builder()
                        .userId(userId)
                        .userName(userName)
                        .syncMessage("Wait for GPS...")
                        .build());
                continue;
            }
```

```
            // Call Geoapify Routing/Distance Service
            DistanceMatrixService.DurationResult duration =
distanceMatrixService.getDuration(
                    loc.getLat(), loc.getLng(), finalDestLat, finalDestLng);

            if (duration != null) {
                // Buffer of 15 mins (900 seconds)
                LocalDateTime recommendedStart =
finalTargetTime.minusSeconds(duration.getSeconds() + 900);

                String msg;
                if (LocalDateTime.now().isAfter(recommendedStart)) {
                    msg = "🔴 Behind schedule";
                } else if (LocalDateTime.now().plusMinutes(30).isAfter(recommendedStart)) {
                    msg = "🟠 Time to start soon";
                } else {
                    msg = "🟢 On track";
                }

                report.getMemberStatus().add(SyncReport.MemberSyncStatus.builder()
                        .userId(userId)
                        .userName(userName)
                        .currentEtaText(duration.getText())
                        .recommendedStartTime(recommendedStart)
                        .syncMessage(msg)
                        .build());
            } else {
                report.getMemberStatus().add(SyncReport.MemberSyncStatus.builder()
                        .userId(userId)
                        .userName(userName)
                        .syncMessage("Routing unavailable")
                        .build());
            }
        }
```

```
        return report;
    }
}
```

File: `notification-service\src\main\java\com\voyzr\notificationservice\VoyzrNotificationServiceApplication.java`

```java
package com.voyzr.notificationservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class VoyzrNotificationServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(VoyzrNotificationServiceApplication.class, args);
    }
}
```

File: `notification-service\src\main\java\com\voyzr\notificationservice\config\JwtAuthFilter.java`

```java
package com.voyzr.notificationservice.config;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.extern.slf4j.Slf4j;
```

```
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.List;

@Component
@Slf4j
public class JwtAuthFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
            throws ServletException, IOException {

        String xUserId = request.getHeader("X-User-Id");
        if (xUserId != null) {
            Long userId = Long.parseLong(xUserId);
            String email = request.getHeader("X-User-Email") != null ?
request.getHeader("X-User-Email") : "user@voyzr.com";

            UsernamePasswordAuthenticationToken authentication =
                    new UsernamePasswordAuthenticationToken(
                            userId,
                            email,
                            List.of(new SimpleGrantedAuthority("ROLE_USER"))
                    );
            SecurityContextHolder.getContext().setAuthentication(authentication);
        }

        filterChain.doFilter(request, response);
```

```
    }
}
```

File: `notification-service\src\main\java\com\voyzr\notificationservice\config\SecurityConfig.java`

```java
package com.voyzr.notificationservice.config;

import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthFilter jwtAuthFilter;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
                .csrf(AbstractHttpConfigurer::disable)
                .authorizeHttpRequests(auth -> auth
                        .anyRequest().permitAll()
```

```
                )
                .sessionManagement(session -> session
                        .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
                )
                .addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}

```

File: `notification-service\src\main\java\com\voyzr\notificationservice\controller\NotificationController.java`

```java
package com.voyzr.notificationservice.controller;

import com.voyzr.notificationservice.entity.Notification;
import com.voyzr.notificationservice.repository.NotificationRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/notifications")
@RequiredArgsConstructor
public class NotificationController {

    private final NotificationRepository notificationRepository;

    @GetMapping("/user/{userId}")
```

```
    public ResponseEntity<List<Notification>> getNotifications(@PathVariable Long userId) {
        return
ResponseEntity.ok(notificationRepository.findByUserIdOrderByTimestampDesc(getCurrentUserId(
)));
    }

    @GetMapping("/user/{userId}/unread-count")
    public ResponseEntity<Long> getUnreadCount(@PathVariable Long userId) {
        return
ResponseEntity.ok(notificationRepository.countByUserIdAndReadFalse(getCurrentUserId()));
    }

    @PutMapping("/{id}/read")
    public ResponseEntity<Notification> markAsRead(@PathVariable Long id) {
        return notificationRepository.findById(id)
                .map(n -> {
                    if (!n.getUserId().equals(getCurrentUserId())) {
                        return
ResponseEntity.status(org.springframework.http.HttpStatus.FORBIDDEN).<Notification>build();
                    }
                    n.setRead(true);
                    return ResponseEntity.ok(notificationRepository.save(n));
                })
                .orElse(ResponseEntity.notFound().build());
    }

    @PutMapping("/user/{userId}/read-all")
    public ResponseEntity<Void> markAllAsRead(@PathVariable Long userId) {
        List<Notification> unread =
notificationRepository.findByUserIdOrderByTimestampDesc(getCurrentUserId());
        unread.forEach(n -> n.setRead(true));
        notificationRepository.saveAll(unread);
        return ResponseEntity.ok().build();
    }

    private Long getCurrentUserId() {
```

```
        org.springframework.security.core.Authentication auth =
org.springframework.security.core.context.SecurityContextHolder.getContext().getAuthenticat
ion();
        if (auth == null || auth.getPrincipal() instanceof String) return 1L; // Fallback
        return (Long) auth.getPrincipal();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteNotification(@PathVariable Long id) {
        notificationRepository.deleteById(id);
        return ResponseEntity.ok().build();
    }
}

```

File: `notification-service\src\main\java\com\voyzr\notificationservice\entity\Notification.java`

```java
package com.voyzr.notificationservice.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "notifications")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Notification {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;

    @Column(nullable = false)
    private Long userId;

    @Column(nullable = false)
    private String title;

    @Column(nullable = false)
    private String message;

    @Column(nullable = false)
    private String type; // trip, chat, expense, system

    @Column(nullable = false)
    private boolean read = false;

    private String actionUrl;

    @Column(nullable = false)
    private LocalDateTime timestamp;

    @PrePersist
    protected void onCreate() {
        timestamp = LocalDateTime.now();
    }
}
```

File: `notification-service\src\main\java\com\voyzr\notificationservice\listener\NotificationListener.java`

```java
package com.voyzr.notificationservice.listener;
```

```
import com.voyzr.notificationservice.entity.Notification;
import com.voyzr.notificationservice.repository.NotificationRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Service;

@Service
@Slf4j
@RequiredArgsConstructor
public class NotificationListener {

    private final NotificationRepository notificationRepository;

    @KafkaListener(topics = "trip-events", groupId = "notification-group")
    public void handleTripEvent(String message) {
        log.info("Received Trip Event: {}", message);

        // Simple parsing or just save the whole message
        Notification notification = Notification.builder()
                .userId(1L) // Default for demo
                .title("Trip Update")
                .message(message)
                .type("trip")
                .read(false)
                .build();
        notificationRepository.save(notification);
    }

    @KafkaListener(topics = "chat-events", groupId = "notification-group")
    public void handleChatEvent(String message) {
        log.info("Received Chat Event: {}", message);

        Notification notification = Notification.builder()
```

```
                .userId(1L) // Default for demo
                .title("New Message")
                .message(message)
                .type("chat")
                .read(false)
                .build();
        notificationRepository.save(notification);
    }
}
```

File: `notification-service\src\main\java\com\voyzr\notificationservice\repository\NotificationRepository.java`

```java
package com.voyzr.notificationservice.repository;

import com.voyzr.notificationservice.entity.Notification;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface NotificationRepository extends JpaRepository<Notification, Long> {
    List<Notification> findByUserIdOrderByTimestampDesc(Long userId);
    long countByUserIdAndReadFalse(Long userId);
}
```

File: `places-service\src\main\java\com\voyzr\placesservice\VoyzrPlacesServiceApplication.java`

```java
package com.voyzr.placesservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class VoyzrPlacesServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(VoyzrPlacesServiceApplication.class, args);
    }
}

```

File: `places-service\src\main\java\com\voyzr\placesservice\controller\PlacesController.java`

```java
package com.voyzr.placesservice.controller;

import com.voyzr.placesservice.entity.Attraction;
import com.voyzr.placesservice.repository.AttractionRepository;
import com.voyzr.placesservice.service.PlacesService;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/places")
@Slf4j
@RequiredArgsConstructor
```

```
public class PlacesController {

    private final PlacesService placesService;
    private final AttractionRepository attractionRepository;

    @GetMapping("/search")
    public List<PlacesService.PlaceSuggestion> search(@RequestParam String query) {
        return placesService.searchPlaces(query);
    }

    @GetMapping("/attractions")
    public List<PlacesService.PlaceSuggestion> getAttractions(@RequestParam double lat,
@RequestParam double lng) {
        return placesService.getAttractions(lat, lng);
    }

    @GetMapping("/all")
    public List<PlacesService.PlaceSuggestion> getAllAttractions() {
        return placesService.getAllAttractions();
    }

    @PostMapping("/attractions/manual")
    public ResponseEntity<Attraction> addManual(@RequestBody Attraction attraction) {
        log.info("SCOUT MODE: Manually capturing intel for {}", attraction.getName());
        return ResponseEntity.ok(attractionRepository.save(attraction));
    }
}

```

File: `places-service\src\main\java\com\voyzr\placesservice\entity\Attraction.java`

```java
package com.voyzr.placesservice.entity;
```

```
import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "attractions")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Attraction {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(length = 1000)
    private String description;

    private String category; // FEAST, LANDMARK, SAFEHOUSE, INTEL

    private String address;

    @Column(name = "lat", nullable = false)
    private Double lat;

    @Column(name = "lng", nullable = false)
    private Double lng;

    private Double rating;

    private String image_url;
```

```
    private boolean is_verified = true;
}
```

File: `places-service\src\main\java\com\voyzr\placesservice\repository\AttractionRepository.java`

```java
package com.voyzr.placesservice.repository;

import com.voyzr.placesservice.entity.Attraction;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface AttractionRepository extends JpaRepository<Attraction, Long> {

    @Query("SELECT a FROM Attraction a WHERE a.lat BETWEEN :minLat AND :maxLat AND a.lng BETWEEN :minLng AND :maxLng")
    List<Attraction> findNear(@Param("minLat") double minLat,
                              @Param("maxLat") double maxLat,
                              @Param("minLng") double minLng,
                              @Param("maxLng") double maxLng);
}
```

File: `places-service\src\main\java\com\voyzr\placesservice\service\GeocodeService.java`

```java
```

```
package com.voyzr.placesservice.service;


import com.fasterxml.jackson.databind.JsonNode;

import com.fasterxml.jackson.databind.ObjectMapper;

import lombok.AllArgsConstructor;

import lombok.Data;

import lombok.extern.slf4j.Slf4j;

import org.springframework.beans.factory.annotation.Value;

import org.springframework.stereotype.Service;

import org.springframework.web.client.RestTemplate;


@Service

@Slf4j

public class GeocodeService {


    @Value("${google.maps.api-key:NOT_SET}")

    private String apiKey;


    private final RestTemplate restTemplate = new RestTemplate();

    private final ObjectMapper objectMapper = new ObjectMapper();


    public GeocodeResult geocode(String address) {

        String url =
String.format("https://maps.googleapis.com/maps/api/geocode/json?address=%s&key=%s",
address,

                apiKey);

        try {

            String response = restTemplate.getForObject(url, String.class);

            JsonNode root = objectMapper.readTree(response);

            if ("OK".equals(root.path("status").asText())) {

                JsonNode loc =
root.path("results").get(0).path("geometry").path("location");

                return new GeocodeResult(loc.path("lat").asDouble(),
loc.path("lng").asDouble());

            }

        } catch (Exception e) {
```

```
            log.error("Geocoding failed: {}", e.getMessage());
        }
        return null;
    }

    @Data
    @AllArgsConstructor
    public static class GeocodeResult {
        private Double latitude;
        private Double longitude;
    }
}

```

File: `places-service\src\main\java\com\voyzr\placesservice\service\PlacesService.java`

```java
package com.voyzr.placesservice.service;

import com.voyzr.placesservice.entity.Attraction;
import com.voyzr.placesservice.repository.AttractionRepository;
import jakarta.annotation.PostConstruct;
import lombok.Data;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

@Service
@Slf4j
@RequiredArgsConstructor
```

```
public class PlacesService {

    private final AttractionRepository attractionRepository;

    @PostConstruct
    public void seedData() {
        log.info("Attraction service initialized. Waiting for database sync.");
    }

    public List<PlaceSuggestion> searchPlaces(String query) {
        return attractionRepository.findAll().stream()
                .filter(a -> a.getName().toLowerCase().contains(query.toLowerCase()))
                .map(this::mapToSuggestion)
                .collect(Collectors.toList());
    }

    public List<PlaceSuggestion> getAttractions(double lat, double lng) {
        log.info("Fetching CUSTOM attractions near {}, {}", lat, lng);

        // Define a bounding box (~10km radius)
        double offset = 0.1; // roughly 11km
        List<Attraction> nearby = attractionRepository.findNear(
                lat - offset, lat + offset,
                lng - offset, lng + offset
        );

        return nearby.stream()
                .map(this::mapToSuggestion)
                .collect(Collectors.toList());
    }

    public List<PlaceSuggestion> getAllAttractions() {
        log.info("Fetching ALL curated database attractions");
        return attractionRepository.findAll().stream()
                .map(this::mapToSuggestion)
```

```
            .collect(Collectors.toList());
    }

    public String getPhotoUrl(String name) {
        return attractionRepository.findAll().stream()
                .filter(a -> a.getName().equalsIgnoreCase(name))
                .findFirst()
                .map(Attraction::getImage_url)
                .orElse("https://loremflickr.com/800/600/travel,landmark");
    }

    private PlaceSuggestion mapToSuggestion(Attraction a) {
        return new PlaceSuggestion(
                a.getName(),
                a.getAddress(),
                a.getLat(),
                a.getLng(),
                a.getRating(),
                a.getName(),
                a.getImage_url()
        );
    }

    @Data
    public static class PlaceSuggestion {
        private String name;
        private String address;
        private Double lat;
        private Double lng;
        private Double rating;
        private String photoReference;
        private String imageUrl;

        public PlaceSuggestion() {}
```

```
        public PlaceSuggestion(String name, String address, Double lat, Double lng, Double
rating, String photoReference, String imageUrl) {
            this.name = name;
            this.address = address;
            this.lat = lat;
            this.lng = lng;
            this.rating = rating;
            this.photoReference = photoReference;
            this.imageUrl = imageUrl;
        }
    }
}
```

File: `trip-service\src\main\java\com\voyzr\tripservice\TripServiceApplication.java`

```java
package com.voyzr.tripservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableAsync;

/**
 * Main entry point for the Voyzr Trip Service.
 *
 * @SpringBootApplication combines three annotations:
 *   - @Configuration: marks this as a source of bean definitions
 *   - @EnableAutoConfiguration: tells Spring Boot to auto-configure based on dependencies
 *   - @ComponentScan: scans this package and sub-packages for Spring components
 */
@SpringBootApplication
@EnableAsync
public class TripServiceApplication {
```

```
    public static void main(String[] args) {
        SpringApplication.run(TripServiceApplication.class, args);
    }
}
```

File: `trip-service\src\main\java\com\voyzr\tripservice\config\DatabaseMigrator.java`

```java
package com.voyzr.tripservice.config;

import com.voyzr.tripservice.entity.Trip;
import com.voyzr.tripservice.entity.TripMember;
import com.voyzr.tripservice.repository.TripRepository;
import com.voyzr.tripservice.repository.TripMemberRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

import java.util.List;
import java.util.UUID;

@Component
@Slf4j
@RequiredArgsConstructor
public class DatabaseMigrator implements CommandLineRunner {

    private final TripRepository tripRepository;
    private final TripMemberRepository tripMemberRepository;

    @Override
    public void run(String... args) {
```

```
        log.info(">>> STARTING DATABASE INTEGRITY CHECK <<<");

        // 1. Fix Trips (Invite Codes)
        List<Trip> allTrips = tripRepository.findAll();
        long tripCount = 0;
        for (Trip trip : allTrips) {
            if (trip.getInviteCode() == null || trip.getInviteCode().isEmpty()) {
                trip.setInviteCode("TRIP-" + UUID.randomUUID().toString().substring(0,
8).toUpperCase());
                tripRepository.save(trip);
                tripCount++;
            }
        }

        // 2. Fix Trip Members (Status)
        List<TripMember> allMembers = tripMemberRepository.findAll();
        long memberCount = 0;
        for (TripMember member : allMembers) {
            if (member.getStatus() == null) {
                member.setStatus(TripMember.MemberStatus.JOINED);
                tripMemberRepository.save(member);
                memberCount++;
            }
        }

        if (tripCount > 0 || memberCount > 0) {
            log.info(">>> SUCCESS: Fixed {} trips and {} members.", tripCount,
memberCount);
        } else {
            log.info(">>> DATABASE IS HEALTHY: No changes needed.");
        }
    }
}
```

File: `trip-service\src\main\java\com\voyzr\tripservice\config\SecurityConfig.java`

```java
package com.voyzr.tripservice.config;

import com.voyzr.tripservice.security.JwtAuthFilter;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.Arrays;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Autowired
    private JwtAuthFilter jwtAuthFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
                .cors(cors -> cors.configurationSource(corsConfigurationSource()))
                .csrf(csrf -> csrf.disable())
```

```
                .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .authorizeHttpRequests(authz -> authz
                        .anyRequest().permitAll()
                )
                .addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOriginPatterns(Arrays.asList("*"));
        config.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "DELETE", "OPTIONS",
"PATCH"));
        config.setAllowedHeaders(Arrays.asList("*"));
        config.setAllowCredentials(true);
        config.setMaxAge(3600L);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        return source;
    }
}

```

File: `trip-service\src\main\java\com\voyzr\tripservice\config\WebSocketConfig.java`

```java
package com.voyzr.tripservice.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.simp.config.MessageBrokerRegistry;
```

```
import org.springframework.web.socket.config.annotation.EnableWebSocketMessageBroker;
import org.springframework.web.socket.config.annotation.StompEndpointRegistry;
import org.springframework.web.socket.config.annotation.WebSocketMessageBrokerConfigurer;
import org.springframework.web.socket.config.annotation.WebSocketTransportRegistration;

@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        config.enableSimpleBroker("/topic");
        config.setApplicationDestinationPrefixes("/app");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws-voyzr")
                .setAllowedOriginPatterns("*")
                .withSockJS();
    }

    @Override
    public void configureWebSocketTransport(WebSocketTransportRegistration registration) {
        registration.setMessageSizeLimit(5 * 1024 * 1024);
        registration.setSendBufferSizeLimit(5 * 1024 * 1024);
        registration.setSendTimeLimit(20 * 1000);
    }
}

```

File: `trip-service\src\main\java\com\voyzr\tripservice\controller\MediaController.java`

```java
```

```
package com.voyzr.tripservice.controller;


import com.voyzr.tripservice.dto.MediaDto;

import com.voyzr.tripservice.dto.TripDto;

import com.voyzr.tripservice.service.MediaService;

import lombok.RequiredArgsConstructor;

import lombok.extern.slf4j.Slf4j;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;


import java.util.List;


@RestController

@RequestMapping("/api/media")

@CrossOrigin(origins = "http://localhost:3000")

@RequiredArgsConstructor

@Slf4j

public class MediaController {


    private final MediaService mediaService;


    @GetMapping("/trip/{tripId}")

    public ResponseEntity<TripDto.ApiResponse<List<MediaDto.MediaResponse>>>
getMediaByTrip(@PathVariable Long tripId) {

        log.info("GET /api/media/trip/{} - fetching media", tripId);

        List<MediaDto.MediaResponse> media = mediaService.getMediaByTrip(tripId);

        return ResponseEntity.ok(TripDto.ApiResponse.success("Media fetched", media));

    }


    @PostMapping

    public ResponseEntity<TripDto.ApiResponse<MediaDto.MediaResponse>>
addMedia(@RequestBody MediaDto.CreateMediaRequest request) {

        log.info("POST /api/media - adding media '{}'", request.getDescription());


        // In real app, get from authentication
```

```
        String userName = "Abhishek VS";


        MediaDto.MediaResponse response = mediaService.addMedia(request, userName);

        return ResponseEntity.ok(TripDto.ApiResponse.success("Media added successfully",
response));

    }

}


```


File: `trip-service\src\main\java\com\voyzr\tripservice\controller\TripController.java`


```java

package com.voyzr.tripservice.controller;


import com.voyzr.tripservice.dto.TripDto;

import com.voyzr.tripservice.service.TripService;

import jakarta.validation.Valid;

import lombok.RequiredArgsConstructor;

import lombok.extern.slf4j.Slf4j;

import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import org.springframework.security.core.Authentication;

import org.springframework.security.core.context.SecurityContextHolder;

import org.springframework.web.bind.annotation.DeleteMapping;

import org.springframework.web.bind.annotation.ExceptionHandler;

import org.springframework.web.bind.annotation.GetMapping;

import org.springframework.web.bind.annotation.PathVariable;

import org.springframework.web.bind.annotation.PostMapping;

import org.springframework.web.bind.annotation.PutMapping;

import org.springframework.web.bind.annotation.RequestBody;

import org.springframework.web.bind.annotation.RequestMapping;

import org.springframework.web.bind.annotation.RequestParam;

import org.springframework.web.bind.annotation.RestController;

import java.util.List;
```

```
@RestController
@RequestMapping("/api/trips")
@RequiredArgsConstructor
@Slf4j
public class TripController {

    private final TripService tripService;

    @ExceptionHandler(Exception.class)
    public ResponseEntity<TripDto.ApiResponse<Void>> handleException(Exception e) {
        log.error("Unhandled exception in TripController: ", e);
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                .body(TripDto.ApiResponse.error(e.getMessage() != null ? e.getMessage() :
"Internal Server Error"));
    }

    @GetMapping("/health")
    public ResponseEntity<String> health() {
        return ResponseEntity.ok("Trip Service is running ✓");
    }

    @GetMapping("/my-trips")
    public ResponseEntity<TripDto.ApiResponse<List<TripDto.TripSummaryResponse>>>
getMyTrips() {
        log.info("GET /api/trips/my-trips");
        return ResponseEntity
                .ok(TripDto.ApiResponse.success("Trips fetched",
tripService.getMyTrips(getCurrentUserId())));
    }

    @GetMapping("/{id}")
    public ResponseEntity<TripDto.ApiResponse<TripDto.TripResponse>>
getTripById(@PathVariable("id") Long id) {
        log.info("GET /api/trips/{}", id);
```

```
        return ResponseEntity.ok(TripDto.ApiResponse.success("Trip fetched",
tripService.getTripById(id)));

    }


    @PostMapping

    public ResponseEntity<TripDto.ApiResponse<TripDto.TripResponse>> createTrip(

            @Valid @RequestBody TripDto.CreateTripRequest request) {

        log.info("POST /api/trips - creating trip: {}", request.getName());

        TripDto.TripResponse trip = tripService.createTrip(request, getCurrentUserId(),
getCurrentUserName());

        return
ResponseEntity.status(HttpStatus.CREATED).body(TripDto.ApiResponse.success("Trip created",
trip));

    }


    @PutMapping("/{id}")

    public ResponseEntity<TripDto.ApiResponse<TripDto.TripResponse>>
updateTrip(@PathVariable("id") Long id,

            @RequestBody TripDto.UpdateTripRequest request) {

        log.info("PUT /api/trips/{}", id);

        return ResponseEntity.ok(

                TripDto.ApiResponse.success("Trip updated", tripService.updateTrip(id,
request, getCurrentUserId())));

    }


    @DeleteMapping("/{id}")

    public ResponseEntity<TripDto.ApiResponse<Void>> deleteTrip(@PathVariable("id") Long
id) {

        log.info("DELETE /api/trips/{}", id);

        tripService.deleteTrip(id, getCurrentUserId());

        return ResponseEntity.ok(TripDto.ApiResponse.success("Trip deleted", null));

    }


    @PostMapping("/{id}/join")

    public ResponseEntity<TripDto.ApiResponse<TripDto.TripResponse>>
joinTrip(@PathVariable("id") Long id) {

        log.info("POST /api/trips/{}/join", id);
```

```
        TripDto.TripResponse trip = tripService.joinTrip(id, getCurrentUserId(),
getCurrentUserName(),
                getCurrentUserEmail());
        return ResponseEntity.ok(TripDto.ApiResponse.success("Joined trip", trip));
    }

    // INVITATIONS
    @PostMapping("/{id}/invite/{userId}")
    public ResponseEntity<TripDto.ApiResponse<Void>> invite(
            @PathVariable("id") Long id,
            @PathVariable("userId") Long userId,
            @RequestBody(required = false) TripDto.InviteRequest inviteRequest) {
        log.info("POST /api/trips/{}/invite/{}", id, userId);
        String userName = (inviteRequest != null && inviteRequest.getUserName() != null)
                ? inviteRequest.getUserName()
                : "Invited User";
        String userEmail = (inviteRequest != null && inviteRequest.getUserEmail() != null)
                ? inviteRequest.getUserEmail()
                : "invited@voyzr.com";
        tripService.inviteUser(id, userId, userName, userEmail);
        return ResponseEntity.ok(TripDto.ApiResponse.success("Invitation sent", null));
    }

    @PostMapping("/{id}/invitation/respond")
    public ResponseEntity<TripDto.ApiResponse<Void>> respond(@PathVariable("id") Long id,
            @RequestParam("accept") boolean accept) {
        log.info("POST /api/trips/{}/invitation/respond?accept={}", id, accept);
        tripService.respondToInvitation(id, getCurrentUserId(), accept);
        return ResponseEntity.ok(TripDto.ApiResponse.success("Response recorded", null));
    }

    @GetMapping("/invitations/pending")
    public ResponseEntity<TripDto.ApiResponse<List<TripDto.TripSummaryResponse>>>
getPending() {
        log.info("GET /api/trips/invitations/pending");
```

```
        List<TripDto.TripSummaryResponse> response =
tripService.getPendingInvitations(getCurrentUserId());

        return ResponseEntity.ok(TripDto.ApiResponse.success("Pending invitations fetched",
response));

    }


    @DeleteMapping("/{tripId}/members/{memberUserId}")

    public ResponseEntity<TripDto.ApiResponse<Void>> removeMember(@PathVariable("tripId")
Long tripId,

            @PathVariable("memberUserId") Long memberUserId) {

        log.info("DELETE /api/trips/{}/members/{}", tripId, memberUserId);

        tripService.removeMember(tripId, memberUserId, getCurrentUserId());

        return ResponseEntity.ok(TripDto.ApiResponse.success("Member removed", null));

    }


    @PutMapping("/{id}/members/{memberId}/role")

    public ResponseEntity<TripDto.ApiResponse<Void>> updateMemberRole(

            @PathVariable("id") Long id,

            @PathVariable("memberId") Long memberId,

            @RequestBody java.util.Map<String, String> request) {

        String role = request.get("role");

        log.info("PUT /api/trips/{}/members/{}/role -> {}", id, memberId, role);

        tripService.updateMemberRole(id, memberId, role, getCurrentUserId());

        return ResponseEntity.ok(TripDto.ApiResponse.success("Role updated", null));

    }


    @PutMapping("/{id}/image")

    public ResponseEntity<TripDto.ApiResponse<Void>> updateTripImage(

            @PathVariable("id") Long id,

            @RequestBody java.util.Map<String, String> request) {

        String imageUrl = request.get("imageUrl");

        log.info("PUT /api/trips/{}/image", id);

        tripService.updateTripImage(id, imageUrl, getCurrentUserId());

        return ResponseEntity.ok(TripDto.ApiResponse.success("Image updated", null));

    }
```

```
    @PostMapping("/join-by-code")
    public ResponseEntity<TripDto.ApiResponse<TripDto.TripResponse>>
joinByCode(@RequestParam("code") String code) {
        log.info("POST /api/trips/join-by-code?code={}", code);
        TripDto.TripResponse trip = tripService.joinTripByCode(code, getCurrentUserId(),
getCurrentUserName(),
                getCurrentUserEmail());
        return ResponseEntity.ok(TripDto.ApiResponse.success("Join request submitted",
trip));
    }

    @PostMapping("/{id}/join-request/{userId}/respond")
    public ResponseEntity<TripDto.ApiResponse<Void>> respondToJoinRequest(
            @PathVariable("id") Long id,
            @PathVariable("userId") Long userId,
            @RequestParam("accept") boolean accept) {
        log.info("POST /api/trips/{}/join-request/{}/respond?accept={}", id, userId,
accept);
        tripService.respondToJoinRequest(id, userId, getCurrentUserId(), accept);
        return ResponseEntity.ok(TripDto.ApiResponse.success("Request processed", null));
    }

    private Long getCurrentUserId() {
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();
        if (auth == null) return 1L; // Fallback for dev
        Object principal = auth.getPrincipal();
        if (principal instanceof Long)
            return (Long) principal;
        if (principal instanceof String) {
            try {
                return Long.parseLong((String) principal);
            } catch (NumberFormatException e) {
                return 1L;
            }
        }
        return 1L;
```

```
    }

    private String getCurrentUserName() {
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();
        if (auth == null || auth.getDetails() == null || !(auth.getDetails() instanceof
String))
            return "Test Admin";
        return (String) auth.getDetails();
    }

    private String getCurrentUserEmail() {
        return "admin@voyzr.com";
    }
}
```

File: `trip-service\src\main\java\com\voyzr\tripservice\dto\MediaDto.java`

```java
package com.voyzr.tripservice.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

public class MediaDto {

    @Data
```

```
    @NoArgsConstructor
    @AllArgsConstructor
    @Builder
    public static class CreateMediaRequest {
        @NotNull(message = "Trip ID is required")
        private Long tripId;

        @NotBlank(message = "Description is required")
        private String description;

        @NotBlank(message = "Image URL is required")
        private String url;

        private String uploadedBy; // Derived from auth or mocked
    }

    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    @Builder
    public static class MediaResponse {
        private Long id;
        private Long tripId;
        private String description;
        private String url;
        private String uploadedBy;
        private LocalDateTime timestamp;
    }
}

```

File: `trip-service\src\main\java\com\voyzr\tripservice\dto\TripDto.java`

```java
```

```
package com.voyzr.tripservice.dto;

import com.fasterxml.jackson.annotation.JsonFormat;
import com.voyzr.tripservice.entity.Trip;
import com.voyzr.tripservice.entity.TripMember;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;

/**
 * DTOs (Data Transfer Objects) are simple classes used to carry data
 * between the client and server.
 */
public class TripDto {

    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    @Builder
    public static class CreateTripRequest {
        @NotBlank(message = "Trip name is required")
        private String name;

        @NotBlank(message = "Destination is required")
        private String destination;

        private Double destinationLat;
```

```
    private Double destinationLng;

    @NotNull(message = "Trip date is required")
    private LocalDate tripDate;

    private LocalDateTime targetArrivalTime;

    private String description;
}

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public static class UpdateTripRequest {
    private String name;
    private String destination;
    private Double destinationLat;
    private Double destinationLng;
    private LocalDate tripDate;
    private LocalDateTime targetArrivalTime;
    private String description;
    private Trip.TripStatus status;
    private com.fasterxml.jackson.databind.JsonNode sideQuests;
}

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public static class TripResponse {
    private Long id;
    private String name;
    private String destination;
    private Double destinationLat;
```

```
    private Double destinationLng;
    private LocalDate tripDate;
    private LocalDateTime targetArrivalTime;
    private String organizerName;
    private Trip.TripStatus status;
    private String description;
    private String imageUrl;
    private String inviteCode;
    private int memberCount;
    private List<MemberResponse> members;
    private LocalDateTime createdAt;
    private com.fasterxml.jackson.databind.JsonNode sideQuests;
}

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public static class TripSummaryResponse {
    private Long id;
    private String name;
    private String destination;
    private Double destinationLat;
    private Double destinationLng;
    private LocalDate tripDate;
    private LocalDateTime targetArrivalTime;
    private Trip.TripStatus status;
    private String imageUrl;
    private String inviteCode;
    private int memberCount;
    private String organizerName;
    private TripMember.MemberRole myRole;
    private com.fasterxml.jackson.databind.JsonNode sideQuests;
}
```

```
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public static class MemberResponse {
    private Long userId;
    private String userName;
    private String userEmail;
    private TripMember.MemberRole role;
    private LocalDateTime joinedAt;
    private Double lat;
    private Double lng;
    private LocalDateTime recommendedStartTime;
    private String eta;
}

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public static class InviteRequest {
    private String userName;
    private String userEmail;
}

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public static class ApiResponse<T> {
    private boolean success;
    private String message;
    private T data;

    public static <T> ApiResponse<T> success(String message, T data) {
```

```
            return
ApiResponse.<T>builder().success(true).message(message).data(data).build();
        }

        public static <T> ApiResponse<T> error(String message) {
            return
ApiResponse.<T>builder().success(false).message(message).data(null).build();
        }
    }

    // DTOs for Expense Splitting
    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    @Builder
    public static class ExpenseSplitRequest {
        @NotNull(message = "Payer user ID is required")
        private Long paidByUserId;

        @NotNull(message = "Split type is required")
        private String splitType; // e.g., "EQUAL", "CUSTOM"

        // Participants and their amounts owed. For EQUAL split, amounts will be
calculated.
        // For CUSTOM, these are directly provided.
        private Map<Long, Double> participantAmounts;

        // Additional fields can be added here as needed, e.g., for notes specific to the
split
    }

    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    @Builder
    public static class SplitDetailsResponse {
```

```
        private String splitId; // Unique ID for this split
        private Long expenseId;
        private Long tripId;
        private Long paidByUserId;
        private String splitType; // e.g., "EQUAL", "CUSTOM"
        private Map<Long, Double> amountsOwed; // userId -> amount owed
        private LocalDateTime splitTimestamp;
        private boolean settled; // Overall settlement status for this split
    }
}
```

File: `trip-service\src\main\java\com\voyzr\tripservice\entity\Media.java`

```java
package com.voyzr.tripservice.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "media")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Media {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
```

```
    private Long tripId;

    @Column(nullable = false)
    private String description; // Added

    @Column(nullable = false, length = 1024) // Longer length for URLs
    private String url;        // Added

    @Column(nullable = false)
    private String uploadedBy;

    @Column(nullable = false)
    private LocalDateTime timestamp; // Renamed from uploadedAt

    @PrePersist
    protected void onCreate() {
        timestamp = LocalDateTime.now();
    }
}

```

File: `trip-service\src\main\java\com\voyzr\tripservice\entity\Trip.java`

```java
package com.voyzr.tripservice.entity;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDate;
import java.time.LocalDateTime;
```

```
import java.util.ArrayList;
import java.util.List;

import org.hibernate.annotations.JdbcTypeCode;
import org.hibernate.type.SqlTypes;
import com.fasterxml.jackson.databind.JsonNode;

/**
 * Trip entity — maps directly to the "trips" table in PostgreSQL.
 *
 * Each field annotated with @Column maps to a database column.
 * @Entity tells JPA/Hibernate to manage this class as a DB table.
 */
@Entity
@Table(name = "trips")
@Data                   // Lombok: generates getters, setters, toString, equals, hashCode
@Builder                // Lombok: enables Trip.builder().name("...").build() pattern
@NoArgsConstructor      // Lombok: generates empty constructor (required by JPA)
@AllArgsConstructor     // Lombok: generates constructor with all fields
public class Trip {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Auto-increment ID
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false)
    private String destination;

    @Column(name = "destination_lat")
    private Double destinationLat;  // Latitude of destination (for Google Maps)

    @Column(name = "destination_lng")
```

```
private Double destinationLng;  // Longitude of destination

@Column(name = "trip_date", nullable = false)
private LocalDate tripDate;

@Column(name = "target_arrival_time")
private LocalDateTime targetArrivalTime;

@Column(name = "organizer_id", nullable = false)
private Long organizerId;       // User ID of whoever created this trip

@Column(name = "organizer_name")
private String organizerName;

// Enum stored as a String in DB (e.g., "UPCOMING", "LIVE", "COMPLETED")
@Enumerated(EnumType.STRING)
@Column(nullable = false)
@Builder.Default
private TripStatus status = TripStatus.UPCOMING;

@Column(columnDefinition = "TEXT")
private String description;

@Column(name = "image_url", columnDefinition = "TEXT")
private String imageUrl;

// One Trip has many TripMembers
// cascade = ALL: if we delete a trip, its members are also deleted
// orphanRemoval = true: if we remove a member from the list, delete from DB
@OneToMany(mappedBy = "trip", cascade = CascadeType.ALL, orphanRemoval = true)
@Builder.Default
private List<TripMember> members = new ArrayList<>();

@Column(name = "created_at", updatable = false)
private LocalDateTime createdAt;
```

```
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @Column(name = "invite_code", unique = true)
    private String inviteCode;

    @JdbcTypeCode(SqlTypes.JSON)
    @Column(name = "side_quests", columnDefinition = "jsonb")
    private JsonNode sideQuests;

    // @PrePersist runs BEFORE the entity is first saved to DB
    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = LocalDateTime.now();
        if (inviteCode == null) {
            inviteCode = "TRIP-" + java.util.UUID.randomUUID().toString().substring(0,
8).toUpperCase();
        }
    }

    // @PreUpdate runs BEFORE every subsequent save
    @PreUpdate
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }

    // Enum for trip status — stored as string in DB
    public enum TripStatus {
        UPCOMING,   // Trip is scheduled but hasn't started
        LIVE,       // Trip is currently happening (members en route)
        COMPLETED,  // Trip is finished
        CANCELLED   // Trip was cancelled
    }
```

```
}
```

File: `trip-service\src\main\java\com\voyzr\tripservice\entity\TripMember.java`

```java
package com.voyzr.tripservice.entity;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

@Entity
@Table(
    name = "trip_members",
    uniqueConstraints = @UniqueConstraint(columnNames = {"trip_id", "user_id"})
)
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class TripMember {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "trip_id", nullable = false)
    private Trip trip;
```

```
@Column(name = "user_id", nullable = false)
private Long userId;

@Column(name = "user_name")
private String userName;

@Column(name = "user_email")
private String userEmail;

@Enumerated(EnumType.STRING)
@Column(nullable = false)
@Builder.Default
private MemberRole role = MemberRole.MEMBER;

@Enumerated(EnumType.STRING)
@Column(nullable = true)
@Builder.Default
private MemberStatus status = MemberStatus.JOINED;

@Column(name = "joined_at", updatable = false)
private LocalDateTime joinedAt;

private Double lat;
private Double lng;

@PrePersist
protected void onCreate() {
    joinedAt = LocalDateTime.now();
}

public enum MemberRole {
    ORGANIZER, CO_ADMIN, MEMBER
}
```

```
    public enum MemberStatus {
        PENDING, JOINED, DECLINED
    }
}

```

File: `trip-service\src\main\java\com\voyzr\tripservice\exception\GlobalExceptionHandler.java`

```java
package com.voyzr.tripservice.exception;

import com.voyzr.tripservice.dto.TripDto;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.stream.Collectors;

// Custom Exception Classes
class TripNotFoundException extends RuntimeException {
    public TripNotFoundException(Long id) {
        super("Trip not found with id: " + id);
    }
}

class UnauthorizedTripActionException extends RuntimeException {
    public UnauthorizedTripActionException(String message) {
        super(message);
    }
```

```
}


class AlreadyMemberException extends RuntimeException {
    public AlreadyMemberException(Long userId, Long tripId) {
        super("User " + userId + " is already a member of trip " + tripId);
    }
}


@RestControllerAdvice
public class GlobalExceptionHandler {


    // Handles JSON mapping errors (e.g. bad date format) -> 400
    @ExceptionHandler(HttpMessageNotReadableException.class)
    public ResponseEntity<TripDto.ApiResponse<Void>>
handleInputError(HttpMessageNotReadableException ex) {
        return ResponseEntity
                .status(HttpStatus.BAD_REQUEST)
                .body(TripDto.ApiResponse.error("JSON Error: " +
ex.getMostSpecificCause().getMessage()));
    }


    // Handles @Valid validation failures -> 400
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<TripDto.ApiResponse<Void>>
handleValidationErrors(MethodArgumentNotValidException ex) {
        String errors = ex.getBindingResult()
                .getFieldErrors()
                .stream()
                .map(FieldError::getDefaultMessage)
                .collect(Collectors.joining(", "));


        return ResponseEntity
                .status(HttpStatus.BAD_REQUEST)
                .body(TripDto.ApiResponse.error("Validation failed: " + errors));
    }
```

```
    @ExceptionHandler(TripNotFoundException.class)

    public ResponseEntity<TripDto.ApiResponse<Void>> handleNotFound(TripNotFoundException
ex) {

        return ResponseEntity

                .status(HttpStatus.NOT_FOUND)

                .body(TripDto.ApiResponse.error(ex.getMessage()));

    }


    @ExceptionHandler(UnauthorizedTripActionException.class)

    public ResponseEntity<TripDto.ApiResponse<Void>>
handleUnauthorized(UnauthorizedTripActionException ex) {

        return ResponseEntity

                .status(HttpStatus.FORBIDDEN)

                .body(TripDto.ApiResponse.error(ex.getMessage()));

    }


    @ExceptionHandler(AlreadyMemberException.class)

    public ResponseEntity<TripDto.ApiResponse<Void>>
handleAlreadyMember(AlreadyMemberException ex) {

        return ResponseEntity

                .status(HttpStatus.BAD_REQUEST)

                .body(TripDto.ApiResponse.error(ex.getMessage()));

    }


    @ExceptionHandler(Exception.class)

    public ResponseEntity<TripDto.ApiResponse<Void>> handleGeneral(Exception ex) {

        return ResponseEntity

                .status(HttpStatus.INTERNAL_SERVER_ERROR)

                .body(TripDto.ApiResponse.error("An unexpected error occurred: " +
ex.getMessage()));

    }

}


```

File: `trip-service\src\main\java\com\voyzr\tripservice\repository\TripMemberRepository.java`

```java
package com.voyzr.tripservice.repository;

import com.voyzr.tripservice.entity.TripMember;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface TripMemberRepository extends JpaRepository<TripMember, Long> {

    List<TripMember> findByTripId(Long tripId);

    boolean existsByTripIdAndUserId(Long tripId, Long userId);

    Optional<TripMember> findByTripIdAndUserId(Long tripId, Long userId);

    List<TripMember> findByUserIdAndStatus(Long userId, TripMember.MemberStatus status);

    int countByTripId(Long tripId);

    int countByTripIdAndStatusIn(Long tripId, java.util.Collection<TripMember.MemberStatus> statuses);
}
```

File: `trip-service\src\main\java\com\voyzr\tripservice\repository\TripRepository.java`

```java
```

```
package com.voyzr.tripservice.repository;

import com.voyzr.tripservice.entity.Trip;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;

/**
 * TripRepository — Spring Data JPA automatically implements this interface.
 * We just define method signatures; Spring generates the SQL queries.
 *
 * JpaRepository<Trip, Long> gives us built-in methods like:
 *   save(), findById(), findAll(), deleteById(), existsById(), count()
 */
@Repository
public interface TripRepository extends JpaRepository<Trip, Long> {

    /**
     * Find all trips where a specific user is a member.
     * Spring translates the method name into a JOIN query automatically.
     *
     * Equivalent SQL:
     *   SELECT DISTINCT t.* FROM trips t
     *   JOIN trip_members m ON t.id = m.trip_id
     *   WHERE m.user_id = :userId
     *   ORDER BY t.trip_date DESC
     */
    @Query("SELECT DISTINCT t FROM Trip t JOIN t.members m WHERE m.userId = :userId AND
(m.status = 'JOINED' OR m.role = 'ORGANIZER') ORDER BY t.tripDate DESC")
    List<Trip> findTripsByMemberUserId(@Param("userId") Long userId);

    /**
```

```
     * Find all trips created by a specific organizer.
     */
    List<Trip> findByOrganizerIdOrderByTripDateDesc(Long organizerId);

    /**
     * Find trips by their status (e.g., all LIVE trips).
     */
    List<Trip> findByStatus(Trip.TripStatus status);

    /**
     * Find a trip by its unique invite code.
     */
    java.util.Optional<Trip> findByInviteCode(String inviteCode);
}
```

File: `trip-service\src\main\java\com\voyzr\tripservice\security\JwtAuthFilter.java`

```java
package com.voyzr.tripservice.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
```

```
import java.util.List;


/**
 * JwtAuthFilter — runs on EVERY incoming HTTP request (before the controller).
 *
 * Flow:
 *   1. Read the "Authorization: Bearer <token>" header
 *   2. Extract and validate the JWT
 *   3. Put user info into Spring's SecurityContext
 *   4. Controllers can then call getCurrentUserId() to know who's logged in
 *
 * OncePerRequestFilter guarantees it only runs once per request.
 */
@Component
@RequiredArgsConstructor
@Slf4j
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtUtil jwtUtil;


    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
            throws ServletException, IOException {


        // 1. Check for Gateway Headers (X-User-Id)
        String xUserId = request.getHeader("X-User-Id");
        if (xUserId != null) {
            Long userId = Long.parseLong(xUserId);
            String email = request.getHeader("X-User-Email") != null ?
request.getHeader("X-User-Email") : "user@voyzr.com";
            String userName = request.getHeader("X-User-Name") != null ?
request.getHeader("X-User-Name") : "Voyzr User";
```

```
            UsernamePasswordAuthenticationToken authentication =
                    new UsernamePasswordAuthenticationToken(
                            userId,
                            email,
                            List.of(new SimpleGrantedAuthority("ROLE_USER"))
                    );
            authentication.setDetails(userName);
            SecurityContextHolder.getContext().setAuthentication(authentication);

            filterChain.doFilter(request, response);
            return;
        }

        // 2. Read the Authorization header
        String authHeader = request.getHeader("Authorization");

        // 3. If no token — inject a hardcoded DEV USER so controllers work without Auth
Service
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            injectDevUser();
            filterChain.doFilter(request, response);
            return;
        }

        // 4. Extract the token (remove "Bearer " prefix)
        String token = authHeader.substring(7);

        // 5. Validate and parse
        if (jwtUtil.isTokenValid(token)) {
            Long userId = jwtUtil.extractUserId(token);
            String userName = jwtUtil.extractUserName(token);
            String email = jwtUtil.extractEmail(token);

            log.debug("Authenticated user: {} (ID: {})", userName, userId);
```

```
        // 6. Create an Authentication object and store it in SecurityContext
        UsernamePasswordAuthenticationToken authentication =
                new UsernamePasswordAuthenticationToken(
                        userId,
                        email,
                        List.of(new SimpleGrantedAuthority("ROLE_USER"))
                );

        authentication.setDetails(userName);
        SecurityContextHolder.getContext().setAuthentication(authentication);
    }

    // 7. Continue to the next filter / controller
    filterChain.doFilter(request, response);
}

/**
 * DEV MODE: Injects a hardcoded user into Spring's SecurityContext.
 *
 * This simulates what the Auth Service would do — it sets:
 *   principal  = userId (Long)   ← used by getCurrentUserId() in controllers
 *   credentials = email (String) ← used by getCurrentUserEmail()
 *   details    = name (String)   ← used by getCurrentUserName()
 *
 * Without this, controllers that call getCurrentUserId() crash with a
 * ClassCastException because Spring's default principal is a String username.
 *
 * ⚠️  Remove or gate behind @Profile("dev") once Auth Service is built.
 */
private void injectDevUser() {
    // These values match the user already in your database
    Long   devUserId = 1L;
    String devName   = "Abhishek VS";
    String devEmail  = "abhishek@voyzr.com";
```

```
        UsernamePasswordAuthenticationToken devAuth =
                new UsernamePasswordAuthenticationToken(
                        devUserId,   // principal  → Long, matches what controllers expect
                        devEmail,    // credentials
                        List.of(new SimpleGrantedAuthority("ROLE_USER"))
                );
        devAuth.setDetails(devName);

        SecurityContextHolder.getContext().setAuthentication(devAuth);
        log.debug("DEV MODE: Injected dev user '{}' (ID: {})", devName, devUserId);
    }
}

```

File: `trip-service\src\main\java\com\voyzr\tripservice\security\JwtUtil.java`

```java
package com.voyzr.tripservice.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import java.security.Key;
import java.util.Date;

/**
 * JwtUtil — reads and validates JWT tokens issued by the Auth Service.
 *
 * The Trip Service never CREATES tokens — only the Auth Service does.
```

```
 * This class only READS them to find out who the user is.
 *
 * How JWT works:
 *   Header.Payload.Signature
 *   The payload contains "claims" like userId, username, email.
 *   We verify the signature using the shared secret key.
 */
@Component
@Slf4j
public class JwtUtil {

    // Injected from application.yml — MUST match the Auth Service's secret
    @Value("${jwt.secret}")
    private String secret;

    /**
     * Parse the token and extract all claims (the payload).
     * Throws an exception automatically if token is expired or tampered with.
     */
    private Claims extractAllClaims(String token) {
        Key key = Keys.hmacShaKeyFor(secret.getBytes());
        return Jwts.parserBuilder()
                .setSigningKey(key)
                .build()
                .parseClaimsJws(token)
                .getBody();
    }

    /** Extract the user's ID from the token (stored as "sub" or custom claim) */
    public Long extractUserId(String token) {
        Claims claims = extractAllClaims(token);
        return Long.parseLong(claims.get("userId", String.class));
    }

    /** Extract the user's display name */
```

```
    public String extractUserName(String token) {
        return extractAllClaims(token).get("name", String.class);
    }

    /** Extract the user's email */
    public String extractEmail(String token) {
        return extractAllClaims(token).getSubject(); // subject is usually email
    }

    /** Check if the token is still valid (not expired, signature matches) */
    public boolean isTokenValid(String token) {
        try {
            Claims claims = extractAllClaims(token);
            return !claims.getExpiration().before(new Date());
        } catch (Exception e) {
            log.error("JWT validation failed: {}", e.getMessage());
            return false;
        }
    }
}
```

File: `trip-service\src\main\java\com\voyzr\tripservice\service\MediaService.java`

```java
package com.voyzr.tripservice.service;

import com.voyzr.tripservice.dto.MediaDto;
import com.voyzr.tripservice.entity.Media;
import com.voyzr.tripservice.repository.MediaRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
```

```
import java.util.List;

import java.util.stream.Collectors;


@Service

@Transactional

@Slf4j

@RequiredArgsConstructor

public class MediaService {


    private final MediaRepository mediaRepository;


    public MediaDto.MediaResponse addMedia(MediaDto.CreateMediaRequest request, String
userName) {

        log.info("Adding media for trip {} by user {}", request.getTripId(), userName);


        Media media = Media.builder()

                .tripId(request.getTripId())

                .description(request.getDescription())

                .url(request.getUrl())

                .uploadedBy(userName)

                .build();


        Media saved = mediaRepository.save(media);

        return toResponse(saved);

    }


    @Transactional(readOnly = true)

    public List<MediaDto.MediaResponse> getMediaByTrip(Long tripId) {

        return mediaRepository.findByTripId(tripId).stream()

                .map(this::toResponse)

                .collect(Collectors.toList());

    }


    private MediaDto.MediaResponse toResponse(Media m) {
```

```
        return MediaDto.MediaResponse.builder()
                .id(m.getId())
                .tripId(m.getTripId())
                .description(m.getDescription())
                .url(m.getUrl())
                .uploadedBy(m.getUploadedBy())
                .timestamp(m.getTimestamp())
                .build();
    }
}
```

File: `trip-service\src\main\java\com\voyzr\tripservice\service\NotificationService.java`

```java
package com.voyzr.tripservice.service;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Service;

@Service
@Slf4j
@RequiredArgsConstructor
public class NotificationService {

    private final KafkaTemplate<String, String> kafkaTemplate;

    @Async
    public void sendTripCreatedEvent(String tripName, String organizerName) {
        try {
```

```
            log.info("Sending Kafka event for trip '{}' created by {}", tripName,
organizerName);
            kafkaTemplate.send("trip-events", "New trip created: " + tripName + " by " +
organizerName);
        } catch (Exception e) {
            log.error("Failed to send Kafka event for trip '{}'", tripName, e);
        }
    }
}
```

File: `trip-service\src\main\java\com\voyzr\tripservice\service\TripService.java`

```java
package com.voyzr.tripservice.service;

import com.voyzr.tripservice.dto.TripDto;
import com.voyzr.tripservice.entity.Trip;
import com.voyzr.tripservice.entity.TripMember;
import com.voyzr.tripservice.repository.TripMemberRepository;
import com.voyzr.tripservice.repository.TripRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;
import java.util.Collections;
import java.util.List;
import java.util.stream.Collectors;

@Service
@Transactional
@Slf4j
```

```
@RequiredArgsConstructor
public class TripService {

    private final TripRepository tripRepository;
    private final TripMemberRepository tripMemberRepository;
    private final NotificationService notificationService;

    public TripDto.TripResponse createTrip(TripDto.CreateTripRequest request, Long userId,
String userName) {
        log.info("Creating trip '{}' for user {}", request.getName(), userId);

        Trip trip = Trip.builder()
                .name(request.getName())
                .destination(request.getDestination())
                .destinationLat(request.getDestinationLat())
                .destinationLng(request.getDestinationLng())
                .tripDate(request.getTripDate())
                .targetArrivalTime(request.getTargetArrivalTime())
                .description(request.getDescription())
                .organizerName(userName)
                .status(Trip.TripStatus.UPCOMING)
                .sideQuests(null)
                .build();

        Trip savedTrip = tripRepository.save(trip);

        TripMember organizer = TripMember.builder()
                .trip(savedTrip)
                .userId(userId)
                .userName(userName)
                .role(TripMember.MemberRole.ORGANIZER)
                .build();
        TripMember savedOrganizer = tripMemberRepository.save(organizer);

        // Notify via NotificationService (asynchronously)
```

```
        notificationService.sendTripCreatedEvent(savedTrip.getName(), userName);


        return toTripResponse(savedTrip, Collections.singletonList(savedOrganizer));
    }


    @Transactional(readOnly = true)
    public TripDto.TripResponse getTripById(Long tripId) {
        return toTripResponse(findTripOrThrow(tripId));
    }


    @Transactional(readOnly = true)
    public List<TripDto.TripSummaryResponse> getMyTrips(Long userId) {
        return tripRepository.findTripsByMemberUserId(userId)
                .stream()
                .map(trip -> toTripSummary(trip, userId))
                .collect(Collectors.toList());
    }


    public TripDto.TripResponse updateTrip(Long tripId, TripDto.UpdateTripRequest request,
Long userId) {
        Trip trip = findTripOrThrow(tripId);
        verifyOrganizer(trip, userId);


        if (request.getName() != null)           trip.setName(request.getName());
        if (request.getDestination() != null)
trip.setDestination(request.getDestination());
        if (request.getDestinationLat() != null)
trip.setDestinationLat(request.getDestinationLat());
        if (request.getDestinationLng() != null)
trip.setDestinationLng(request.getDestinationLng());
        if (request.getTripDate() != null)       trip.setTripDate(request.getTripDate());
        if (request.getTargetArrivalTime() != null)
trip.setTargetArrivalTime(request.getTargetArrivalTime());
        if (request.getDescription() != null)
trip.setDescription(request.getDescription());
        if (request.getStatus() != null)         trip.setStatus(request.getStatus());
```

```
        if (request.getSideQuests() != null)
trip.setSideQuests(request.getSideQuests());


        return toTripResponse(tripRepository.save(trip));

    }


    public void deleteTrip(Long tripId, Long userId) {

        Trip trip = findTripOrThrow(tripId);

        verifyOrganizer(trip, userId);

        tripRepository.delete(trip);

    }


    public TripDto.TripResponse joinTrip(Long tripId, Long userId, String userName, String
userEmail) {

        Trip trip = findTripOrThrow(tripId);

        TripMember member = TripMember.builder()

                .trip(trip)

                .userId(userId)

                .userName(userName)

                .userEmail(userEmail)

                .role(TripMember.MemberRole.MEMBER)

                .status(TripMember.MemberStatus.JOINED)

                .build();

        tripMemberRepository.save(member);

        return toTripResponse(findTripOrThrow(tripId));

    }


    public void inviteUser(Long tripId, Long userId, String userName, String userEmail) {

        Trip trip = findTripOrThrow(tripId);

        if (tripMemberRepository.findByTripIdAndUserId(tripId, userId).isPresent()) {

            throw new RuntimeException("User already in trip");

        }

        TripMember member = TripMember.builder()

                .trip(trip)

                .userId(userId)
```

```
                .userName(userName)
                .userEmail(userEmail)
                .role(TripMember.MemberRole.MEMBER)
                .status(TripMember.MemberStatus.PENDING)
                .build();
        tripMemberRepository.save(member);
    }

    public void respondToInvitation(Long tripId, Long userId, boolean accept) {
        TripMember member = tripMemberRepository.findByTripIdAndUserId(tripId, userId)
                .orElseThrow(() -> new RuntimeException("Invitation not found"));

        if (member.getStatus() != TripMember.MemberStatus.PENDING) {
            throw new RuntimeException("Invitation already processed");
        }

        if (accept) {
            member.setStatus(TripMember.MemberStatus.JOINED);
            tripMemberRepository.save(member);
        } else {
            tripMemberRepository.delete(member);
        }
    }

    public List<TripDto.TripSummaryResponse> getPendingInvitations(Long userId) {
        // This requires a repository method to find by userId and PENDING status
        return tripMemberRepository.findByUserIdAndStatus(userId,
TripMember.MemberStatus.PENDING)
                .stream()
                .map(m -> toTripSummary(m.getTrip(), userId))
                .collect(Collectors.toList());
    }

    public void removeMember(Long tripId, Long memberUserId, Long requestingUserId) {
        Trip trip = findTripOrThrow(tripId);
```

```
        boolean isSelf = memberUserId.equals(requestingUserId);
        boolean isOrganizer = trip.getOrganizerId().equals(requestingUserId);

        if (!isSelf && !isOrganizer) {
            throw new RuntimeException("Only the organizer can remove other members");
        }

        TripMember member = tripMemberRepository.findByTripIdAndUserId(tripId,
memberUserId)
                .orElseThrow(() -> new RuntimeException("Member not found in this trip"));

        if (member.getRole() == TripMember.MemberRole.ORGANIZER) {
            throw new RuntimeException("The organizer cannot be removed from the trip");
        }

        tripMemberRepository.delete(member);
        log.info("User {} removed from trip {} by user {}", memberUserId, tripId,
requestingUserId);
    }

    @Transactional
    public void updateMemberRole(Long tripId, Long memberUserId, String roleStr, Long
requesterId) {
        Trip trip = findTripOrThrow(tripId);

        TripMember requester = tripMemberRepository.findByTripIdAndUserId(tripId,
requesterId)
                .orElseThrow(() -> new RuntimeException("Not a member of this trip"));

        if (requester.getRole() != TripMember.MemberRole.ORGANIZER) {
            throw new RuntimeException("Only organizers can change roles");
        }

        TripMember member = tripMemberRepository.findByTripIdAndUserId(tripId,
memberUserId)
```

```
                .orElseThrow(() -> new RuntimeException("Member not found in this trip"));

        member.setRole(TripMember.MemberRole.valueOf(roleStr));
        tripMemberRepository.save(member);
    }

    @Transactional
    public void updateTripImage(Long tripId, String imageUrl, Long requesterId) {
        Trip trip = findTripOrThrow(tripId);

        TripMember requester = tripMemberRepository.findByTripIdAndUserId(tripId,
requesterId)
                .orElseThrow(() -> new RuntimeException("Not a member of this trip"));

        if (requester.getRole() != TripMember.MemberRole.ORGANIZER && requester.getRole()
!= TripMember.MemberRole.CO_ADMIN) {
            throw new RuntimeException("Only organizers or co-admins can update group
photo");
        }

        trip.setImageUrl(imageUrl);
        tripRepository.save(trip);
    }

    public TripDto.TripResponse joinTripByCode(String inviteCode, Long userId, String
userName, String userEmail) {
        Trip trip = tripRepository.findByInviteCode(inviteCode)
                .orElseThrow(() -> new RuntimeException("Trip not found with code: " +
inviteCode));

        if (tripMemberRepository.findByTripIdAndUserId(trip.getId(), userId).isPresent()) {
            throw new RuntimeException("You are already a member of this trip");
        }

        TripMember member = TripMember.builder()
                .trip(trip)
```

```
                .userId(userId)
                .userName(userName)
                .userEmail(userEmail)
                .role(TripMember.MemberRole.MEMBER)
                .status(TripMember.MemberStatus.PENDING)
                .build();
        tripMemberRepository.save(member);

        // Notify organizer via Kafka (optional)
        return toTripResponse(trip);
    }

    public void respondToJoinRequest(Long tripId, Long memberUserId, Long organizerId,
boolean accept) {
        Trip trip = findTripOrThrow(tripId);
        verifyOrganizer(trip, organizerId);

        TripMember member = tripMemberRepository.findByTripIdAndUserId(tripId,
memberUserId)
                .orElseThrow(() -> new RuntimeException("Request not found"));

        if (member.getStatus() != TripMember.MemberStatus.PENDING) {
            throw new RuntimeException("Request already processed");
        }

        if (accept) {
            member.setStatus(TripMember.MemberStatus.JOINED);
            tripMemberRepository.save(member);
        } else {
            tripMemberRepository.delete(member);
        }
    }

    private Trip findTripOrThrow(Long tripId) {
        return tripRepository.findById(tripId)
```

```
                .orElseThrow(() -> new RuntimeException("Trip not found: " + tripId));
    }

    private void verifyOrganizer(Trip trip, Long userId) {
        if (!trip.getOrganizerId().equals(userId)) {
            throw new RuntimeException("Unauthorized: Only organizer can do this");
        }
    }

    private TripDto.TripResponse toTripResponse(Trip trip) {
        List<TripMember> members = tripMemberRepository.findByTripId(trip.getId());
        return toTripResponse(trip, members);
    }

    private TripDto.TripResponse toTripResponse(Trip trip, List<TripMember> members) {
        List<TripDto.MemberResponse> memberResponses = members
                .stream()
                .filter(m -> m.getStatus() == TripMember.MemberStatus.JOINED || m.getRole()
== TripMember.MemberRole.ORGANIZER)
                .map(m -> {
                    LocalDateTime recommendedStart = null;
                    String etaText = "Calculating...";

                    if (trip.getTargetArrivalTime() != null) {
                        recommendedStart = trip.getTargetArrivalTime().minusHours(2);
                        etaText = "~2h";
                    }

                    return TripDto.MemberResponse.builder()
                        .userId(m.getUserId())
                        .userName(m.getUserName())
                        .userEmail(m.getUserEmail())
                        .role(m.getRole())
                        .joinedAt(m.getJoinedAt())
                        .lat(m.getLat())
```

```
                .lng(m.getLng())
                .recommendedStartTime(recommendedStart)
                .eta(etaText)
                .build();
        })
        .collect(Collectors.toList());

    return TripDto.TripResponse.builder()
        .id(trip.getId())
        .name(trip.getName())
        .destination(trip.getDestination())
        .destinationLat(trip.getDestinationLat())
        .destinationLng(trip.getDestinationLng())
        .tripDate(trip.getTripDate())
        .targetArrivalTime(trip.getTargetArrivalTime())
        .organizerName(trip.getOrganizerName())
        .status(trip.getStatus())
        .description(trip.getDescription())
        .imageUrl(trip.getImageUrl())
        .inviteCode(trip.getInviteCode())
        .memberCount(memberResponses.size())
        .members(memberResponses)
        .createdAt(trip.getCreatedAt())
        .sideQuests(trip.getSideQuests())
        .build();
}

private TripDto.TripSummaryResponse toTripSummary(Trip trip, Long currentUserId) {
    TripMember.MemberRole myRole = tripMemberRepository
        .findByTripIdAndUserId(trip.getId(), currentUserId)
        .map(TripMember::getRole)
        .orElse(TripMember.MemberRole.MEMBER);

    return TripDto.TripSummaryResponse.builder()
        .id(trip.getId())
```

```
                .name(trip.getName())
                .destination(trip.getDestination())
                .destinationLat(trip.getDestinationLat())
                .destinationLng(trip.getDestinationLng())
                .tripDate(trip.getTripDate())
                .targetArrivalTime(trip.getTargetArrivalTime())
                .status(trip.getStatus())
                .imageUrl(trip.getImageUrl())
                .inviteCode(trip.getInviteCode() != null ? trip.getInviteCode() : "N/A")
                .memberCount(tripMemberRepository.countByTripIdAndStatusIn(trip.getId(),
List.of(TripMember.MemberStatus.JOINED)))
                .organizerName(trip.getOrganizerName())
                .myRole(myRole)
                .sideQuests(trip.getSideQuests())
                .build();
    }
}
```

File: `trip-service\src\test\java\com\voyzr\tripservice\TripServiceTest.java`

```java
package com.voyzr.tripservice;

import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;
import com.voyzr.tripservice.dto.TripDto;
import com.voyzr.tripservice.entity.Trip;
import com.voyzr.tripservice.repository.TripMemberRepository;
import com.voyzr.tripservice.repository.TripRepository;
import com.voyzr.tripservice.service.TripService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.ActiveProfiles;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDate;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

/**
 * Integration tests for TripService.
 * @SpringBootTest loads the full application context.
 * @ActiveProfiles("test") uses application-test.yml (H2 in-memory DB).
 * @Transactional rolls back DB changes after each test so tests are isolated.
 */
@SpringBootTest
@Transactional
@ActiveProfiles("test")
class TripServiceTest {

    @Autowired private TripService tripService;
    @Autowired private TripRepository tripRepository;
    @Autowired private TripMemberRepository tripMemberRepository;

    private TripDto.CreateTripRequest validRequest;
    private final Long USER_ID = 1L;
    private final String USER_NAME = "Abhishek VS";

    @BeforeEach
    void setUp() {
        // Build a valid create request used by most tests
        validRequest = TripDto.CreateTripRequest.builder()
                .name("Munnar Weekend")
                .destination("Munnar, Kerala")
```

```
                .destinationLat(10.0889)
                .destinationLng(77.0595)
                .tripDate(LocalDate.now().plusDays(7))
                .description("Tea estate visit!")
                .build();
    }

    @Test
    void createTrip_shouldSaveTripAndAddOrganizerAsMember() {
        TripDto.TripResponse response = tripService.createTrip(validRequest, USER_ID,
USER_NAME);

        assertNotNull(response.getId(), "Trip should get an ID after saving");
        assertEquals("Munnar Weekend", response.getName());
        assertEquals("Munnar, Kerala", response.getDestination());
        assertEquals(Trip.TripStatus.UPCOMING, response.getStatus());
        assertEquals(1, response.getMemberCount(), "Creator should be auto-added as
member");
        assertEquals(USER_NAME, response.getOrganizerName());
    }

    @Test
    void getMyTrips_shouldReturnTripsWhereUserIsMember() {
        // Create two trips for the same user
        tripService.createTrip(validRequest, USER_ID, USER_NAME);

        TripDto.CreateTripRequest anotherTrip = TripDto.CreateTripRequest.builder()
                .name("Varkala Beach")
                .destination("Varkala")
                .tripDate(LocalDate.now().plusDays(14))
                .build();
        tripService.createTrip(anotherTrip, USER_ID, USER_NAME);

        List<TripDto.TripSummaryResponse> myTrips = tripService.getMyTrips(USER_ID);
        assertEquals(2, myTrips.size(), "Should return both trips for this user");
```

```
    }

    @Test
    void joinTrip_shouldAddNewMember() {
        TripDto.TripResponse created = tripService.createTrip(validRequest, USER_ID,
USER_NAME);

        Long newUserId = 2L;
        TripDto.TripResponse afterJoin = tripService.joinTrip(
                created.getId(), newUserId, "Adil Navas", "adil@example.com");

        assertEquals(2, afterJoin.getMemberCount(), "Should now have 2 members");
    }

    @Test
    void joinTrip_shouldThrowIfAlreadyMember() {
        TripDto.TripResponse created = tripService.createTrip(validRequest, USER_ID,
USER_NAME);

        // Try to join again as the same user (already organizer/member)
        assertThrows(Exception.class, () ->
                tripService.joinTrip(created.getId(), USER_ID, USER_NAME, "ab@email.com"),
                "Should throw when user is already a member"
        );
    }

    @Test
    void deleteTrip_shouldFailIfNotOrganizer() {
        TripDto.TripResponse created = tripService.createTrip(validRequest, USER_ID,
USER_NAME);

        Long differentUser = 99L;
        assertThrows(Exception.class, () ->
                tripService.deleteTrip(created.getId(), differentUser),
                "Non-organizer should not be able to delete trip"
        );
```

```
    }
}
```

File: `user-service\src\main\java\com\voyzr\userservice\VoyzrUserServiceApplication.java`

```java
package com.voyzr.userservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.security.crypto.password.PasswordEncoder;
import com.voyzr.userservice.entity.User;
import com.voyzr.userservice.repository.UserRepository;

@SpringBootApplication
public class VoyzrUserServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(VoyzrUserServiceApplication.class, args);
    }

    @Bean
    CommandLineRunner seedUsers(UserRepository userRepository, PasswordEncoder passwordEncoder) {
        return args -> {
            if (userRepository.count() == 0) {
                userRepository.save(User.builder()
                    .username("voyzr_admin")
                    .email("admin@voyzr.com")
                    .password(passwordEncoder.encode("admin123"))
```

```
                    .firstName("Voyzr")
                    .lastName("Admin")
                    .build());
                userRepository.save(User.builder()
                    .username("john_doe")
                    .email("john@voyzr.com")
                    .password(passwordEncoder.encode("password123"))
                    .firstName("John")
                    .lastName("Doe")
                    .build());
                userRepository.save(User.builder()
                    .username("alice_traveler")
                    .email("alice@voyzr.com")
                    .password(passwordEncoder.encode("password123"))
                    .firstName("Alice")
                    .lastName("Smith")
                    .build());
                System.out.println("✓ Seeded 3 test users with hashed passwords.");
                System.out.println("  → Login: admin@voyzr.com / admin123");
                System.out.println("  → Login: john@voyzr.com / password123");
                System.out.println("  → Login: alice@voyzr.com / password123");
            } else {
                System.out.println("✓ Users already exist in DB, skipping seed.");
            }
        };
    }
}
```

File: `user-service\src\main\java\com\voyzr\userservice\config\ApplicationConfig.java`

```java
package com.voyzr.userservice.config;
```

```
import com.voyzr.userservice.repository.UserRepository;

import lombok.RequiredArgsConstructor;

import org.springframework.context.annotation.Bean;

import org.springframework.context.annotation.Configuration;

import org.springframework.security.authentication.AuthenticationManager;

import org.springframework.security.authentication.AuthenticationProvider;

import org.springframework.security.authentication.dao.DaoAuthenticationProvider;

import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationC
onfiguration;

import org.springframework.security.core.userdetails.UserDetailsService;

import org.springframework.security.core.userdetails.UsernameNotFoundException;

import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;

import org.springframework.security.crypto.password.PasswordEncoder;


import java.util.Collections;


@Configuration

@RequiredArgsConstructor

public class ApplicationConfig {


    private final UserRepository userRepository;


    @Bean

    public UserDetailsService userDetailsService() {

        return username -> userRepository.findByEmail(username)

                .map(user -> new org.springframework.security.core.userdetails.User(

                        user.getEmail(),

                        user.getPassword(),

                        Collections.emptyList()

                ))

                .orElseThrow(() -> new UsernameNotFoundException("User not found"));

    }
```

```
    @Bean
    public AuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
        authProvider.setUserDetailsService(userDetailsService());
        authProvider.setPasswordEncoder(passwordEncoder());
        return authProvider;
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config)
throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}

```